# On-Prem Data Fabric Architecture





- Author: Mark Raysmith Cityline Ltd

- Created: 2025-10-20
- Last updated: 2026-08-25

## 1. Executive Summary

This document defines a pragmatic on-prem data fabric architecture built around **Ceph RGW object storage**, **Apache Iceberg**, **Apache Spark**, **Apache Flink**, a **central REST-oriented Iceberg catalog**, **Apache Atlas**, **Apache Ranger**, **Apache Airflow**, **Trino**, and an optional **Kafka-backed event backbone** plus an optional serving layer (**Firebolt Core** or **ClickHouse**).

The design goal is not to assemble a random list of fashionable tools. The goal is to create a governed, scalable, batch-and-streaming-capable platform that supports:

- enterprise data ingestion
- streaming and batch processing
- open table semantics
- strong metadata and lineage
- low-latency SQL serving for curated data products
- on-prem deployment and control

The recommended architectural position is:

- **Ceph RGW object storage** is the persistence layer
- **Iceberg** is the mandatory table abstraction and data contract
- **Spark** is the primary batch compute engine
- **Flink** is the primary streaming and real-time compute engine
- **Kafka** is the recommended event backbone when the platform requires durable, replayable shared streaming and CDC ingestion
- **Apache Polaris** is the chosen REST catalog implementation and metadata control point for multi-engine interoperability
- **Atlas** is the metadata and governance plane
- **Ranger** is the policy enforcement layer for classification-driven access control
- **Airflow** is the orchestration and control-plane scheduler
- **Trino** is the default shared interactive SQL query plane over governed Iceberg datasets

- **Firebolt Core** or **ClickHouse** is an optional acceleration layer for interactive analytics over Iceberg-backed data; the engine choice is a Phase 3 evaluation decision

The most important decision in this design is to make **Apache Iceberg the foundational abstraction**. Without that, the platform degenerates into a file swamp. Apache Iceberg is explicitly designed as a high-performance table format for large analytic datasets and supports safe multi-engine access from engines including Spark and Flink. The catalog choice is also a first-order design decision, not a later implementation detail. This architecture standardizes on a centrally managed REST-oriented catalog to reduce cross-engine ambiguity. See the Iceberg documentation and multi-engine support references:

- Apache Iceberg documentation: https://iceberg.apache.org/docs/latest/
- Apache Iceberg overview: https://iceberg.apache.org/
- Multi-engine support: https://iceberg.apache.org/multi-engine-support/

---

# 2. Architecture Principles

## 2.1 Open formats over proprietary lock-in

All persisted analytical datasets should be stored in open formats using **Apache Iceberg tables** over files in object storage.

## 2.2 Storage and table semantics must be separated

Ceph RGW provides S3-compatible object storage for files. It is **not** the semantic contract for consumers. The semantic contract is the **Iceberg table**.

## 2.3 Streaming and batch are separate compute concerns

- **Flink** handles continuous, stateful, event-driven, near-real-time processing
- **Spark** handles scheduled, bounded, heavy compute and historical reshaping, including micro-batch execution where a freshness requirement is measured in minutes rather than seconds

Trying to force one engine to do both jobs badly is poor architecture.

This separation is about compute. It does not decide how data lands in a table: both engines commit to the same Iceberg tables through the same commit protocol, and the write mode is a separate per-table decision defined in §6.4.

## 2.4 Governance is a first-class platform capability

A data fabric without cataloguing, ownership, lineage, classification, and enforceable policy is just storage plus processing. Governance must be built in from day one via **Apache Atlas**, **Apache Ranger**, and enforced naming, ownership, metadata publication, and classification conventions.

## 2.5 Orchestration is not streaming

**Airflow** should orchestrate finite, bounded workflows and platform operations. It should not be used as a streaming runtime or a pseudo event bus. Apache Airflow is explicitly a workflow platform for authoring, scheduling, and monitoring workflows as directed acyclic graphs of tasks:

- Apache Airflow overview: https://airflow.apache.org/

## 2.6 Query serving is structured in two layers

The default shared query plane should be **Trino over Iceberg** for open interactive SQL access. If deployed, the acceleration engine (**Firebolt Core** or **ClickHouse**) should sit northbound of curated Iceberg datasets as an optional acceleration and serving layer, not as the foundational storage or governance layer.

## 2.7 Platform behavior must be verified by contracts

Every platform layer should expose a small set of explicit contracts that can be verified automatically: storage buckets and policies, catalog namespaces, table schemas, Spark job outcomes, Airflow DAG behavior, quality gate decisions, query results, lineage publication, and access control. The platform is not considered healthy because services are running; it is healthy when those contracts pass against the live stack.

### 2.7.1 Two implementation profiles

Every phase and increment is implemented through two explicit tracks. They share service contracts, schemas, protocols, verifier artifacts, and functional tests, but they do not claim the same operational evidence.

| Track | Purpose | Permitted simplifications | Required evidence |
|---|---|---|---|
| Developer profile | fast, repeatable engineering on a workstation or small integration host | Docker Desktop or Podman, reduced replicas, local bind mounts or named volumes, local CA certificates, loopback/private-network HTTP where explicitly permitted, and disposable state | idempotent startup/shutdown, health checks, functional contract tests, pinned artifacts, and a documented reset path |
| Production profile | durable on-prem service for governed datasets | only exceptions recorded with an owner, risk, expiry or retest trigger, and approved remediation | supported distributed topology, durable shared state, trusted TLS, managed service identities and secrets, authorization, observability, backup/restore, failure drills, capacity evidence, and operational ownership |

A developer-profile gate proves that engineers can build and exercise the capability. It never proves production readiness. Production promotion must replace every developer-only shortcut and rerun the shared functional suite plus the production security, durability, recovery, and operations tests. An increment that cannot identify the replacement for a shortcut is incomplete.

The version policy applies to both tracks: select the latest compatible supported release, pin the exact patch and artifact digest, and retest the compatibility set together. An older pin requires a written compatibility reason, owner, test evidence, and upgrade trigger; release age alone is not a reason to destabilize a validated integration.

---

## 2.8 Storage architecture decision

The storage platform decision belongs at the architecture level because every downstream increment depends on it. Increment 1 implements the selected storage target; it does not own the architecture tradeoff.

The storage decision is not approved merely because a product is named. Qualification is staged so that the architecture decision does not depend on components that have not yet been delivered:

- **candidate admission before Increment 1** uses documented requirements, release and deployment review, operational fit, known compatibility evidence, and explicit proof targets
- **Increment 1 acceptance** proves the deployed storage service independently through S3 protocol, security, health, failure, synthetic load, capacity, and operational tests
- **cross-increment qualification** proves each real engine when that engine is introduced
- **Phase 1 readiness** proves the complete concurrent engine workload and production operating model

The decision process is:

1. Confirm the Stratus storage requirements.
2. Screen plausible open-source, on-prem storage candidates.
3. Compare the viable candidates against the requirements.
4. Define the staged proof-of-fit targets and admit one candidate into Increment 1.
5. Record any gaps, mitigations, and disqualifying risks.
6. Only then proceed into the Increment 1 implementation runbook.

This section exists because "S3-compatible" is not specific enough for a production data-fabric storage decision. Stratus needs a storage platform that works with Iceberg, Polaris, Spark, Airflow, Trino, and future Flink workloads, and that can be operated on premises with a real production recovery model.

#### 2.8.1 Stratus storage requirements

| Requirement | Why it matters | Acceptance evidence |
|---|---|---|
| Open-source on-prem deployment | Stratus must not depend on a proprietary managed object store for the foundation layer. | License and deployment model recorded for the selected release. |
| Production distributed storage | The storage layer must survive node/disk failure and support capacity growth. | HA topology, failure-domain design, and recovery drills documented. |
| S3-compatible client contract | Polaris, Iceberg, Spark, Trino, Airflow, and Java verification use an S3-style object API. | Required S3 operations pass through the verification suite. |
| Iceberg table safety | Iceberg writes metadata and data files and relies on predictable read-after-write/list behavior. | Iceberg create/write/read/snapshot/maintenance tests pass through Polaris and Spark. |
| Multipart upload behavior | Iceberg and Spark may use multipart upload for larger parquet files. | Multipart create, complete, abort, list parts, and retry behavior verified. |
| Concurrent engine access | Spark, Trino, Polaris, and Airflow can all touch the storage layer during normal operation. | Concurrent read/write/list tests run without throttling, stale reads, or authorization leakage. |

| Requirement | Why it matters | Acceptance evidence |
|---|---|---|
| Metadata-heavy Iceberg behavior | Iceberg creates many metadata files, manifests, and object listings as tables grow. | Object count, listing latency, bucket-index behavior, and metadata file growth are tested with representative table counts. |
| Large scan and ingestion throughput | Storage must sustain initial batch ingestion and analytical scans without becoming the first bottleneck. | Baseline throughput, request latency, request error rate, and retry rate are recorded under Spark and Trino smoke loads. |
| Path-style endpoint support | Internal DNS and lab deployments often use endpoint overrides rather than virtual-hosted buckets. | Spark, Trino, Java SDK, and Polaris configs work with `S3_PATH_STYLE_ACCESS=true`. |
| Service identity isolation | Platform services must not share one storage credential. | `svc-spark`, `svc-polaris`, `svc-airflow`, and `svc-trino` access tests pass and cross-bucket denies are proven. |
| TLS and CA trust | Production traffic must not rely on insecure TLS bypass. | HTTPS endpoint works with trusted CA; plaintext and untrusted connections fail. |
| Encryption-at-rest path | Gold/platform data require an approved at-rest protection model. | Storage encryption design is selected and tested for the chosen target. |
| Operational observability | Operators need health, capacity, request/error, and recovery visibility. | Dashboard/CLI/metrics checks are part of acceptance. |
| Backup, recovery, and failure drills | Production readiness requires proof that loss and recovery procedures work. | Disk/node/gateway failure drills and restore tests complete successfully. |
| Upgrade and lifecycle model | The storage platform will need patching and upgrades after Phase 1. | Release pinning, upgrade path, and rollback constraints are documented. |
| Cost and operating model | On-prem storage shifts cost into hardware, power, capacity planning, and operator effort. | Capacity model, growth assumptions, replication/erasure-coding overhead, and operational ownership are recorded before production onboarding. |
| Governance integration path | Increment 6 adds Ranger/Atlas; storage must not block identity and policy integration. | Authz boundary is defined: storage service credentials at layer 1; analytical user policy through Polaris/Trino/Ranger later. |

#### 2.8.1.1 Phase 1 integrated storage qualification evidence

The storage decision must define measurable proof targets, not only product features. The following matrix is accumulated across Phase 1 as the owning engines become available; it is not the Increment 1 exit gate and is not a prerequisite for starting Increment 2. Exact numeric targets are environment-specific, but each run must record the target, observed result, error budget, test dataset size, object count, concurrency level, hardware profile, and operator effort.

| Evidence area | Required workload | Required metrics | Minimum acceptance rule |
|---|---|---|---|
| Concurrent engine access | Spark writes Iceberg data, Trino reads an existing table, Polaris resolves namespaces/tables, and an operator S3 client lists representative prefixes at the same time. | p50/p95/p99 request latency, 4xx/5xx rate, retry rate, stale-read incidents, authz failures, failed Iceberg commits. | No stale reads, no authorization leakage, no failed committed writes, and request/error metrics remain within the pre-declared smoke-test threshold. |
| Large scan/read throughput | Trino scans a scaled gold table while Spark reads the same table or a representative silver table. | Sustained read throughput, query elapsed time, request latency, retry rate, RGW/S3 gateway CPU/network saturation. | Throughput baseline is recorded and the object store is not the first saturated component unless explicitly accepted with a capacity plan. |
| Ingestion/write throughput | Spark writes a scaled bronze/silver dataset using the same S3 client settings intended for production. | Sustained write throughput, multipart upload success/abort behavior, commit duration, request error rate, retry count. | Writes complete without multipart leaks, failed commits, or elevated retry/error rates beyond the declared threshold. |
| Metadata-heavy listing behavior | Create representative Iceberg table layouts with many snapshots, manifests, partition prefixes, and metadata files. | List latency by prefix, object count, bucket-index health, metadata-file growth, manifest count, snapshot chain length. | Listing and metadata resolution remain predictable enough for maintenance and query planning; any degraded behavior has a documented object-count threshold and mitigation. |
| Small-file/object-count stress | Generate small-file debt, then run compaction and orphan cleanup through Iceberg maintenance. | Object count before/after, average file size, compaction | Maintenance reduces file-count debt and does not destabilize |

| Evidence area | Required workload | Required metrics | Minimum acceptance rule |
|---|---|---|---|
| | | duration, orphan count, delete-file count, request latency/error rate during maintenance. | concurrent reads or catalog operations. |
| Request latency and error budget | Run mixed read/write/list/head/delete/multipart operations through the selected S3 endpoint. | p50/p95/p99 latency by operation, 4xx/5xx rate, timeout rate, retry rate. | The owning gate must meet its declared smoke-test SLO or produce an accepted remediation and retest plan. |
| Cost and capacity model | Model usable capacity for the selected replication/erasure-coding profile and expected growth. | Raw-to-usable ratio, metadata overhead, bucket-index overhead, projected 12/24/36-month capacity, power/rack assumptions where known. | Capacity model shows usable headroom for onboarding and defines expansion triggers before production data is accepted. |
| Operator effort | Execute install, upgrade rehearsal, failure drill, restore drill, credential rotation, and dashboard/alert setup. | Operator steps, elapsed operator time, specialist skills, automation gaps, runbook defects. | Operational burden is recorded and accepted by the owning operations team, including any staffing or automation gaps. |

#### 2.8.2 Candidate screen

The initial open-source on-prem candidate set is:

| Candidate | Advanced to detailed fit? | Reason |
|---|---|---|
| Ceph RGW | Yes | Mature open-source distributed storage platform with S3-compatible RGW, replication/erasure coding, dashboard, health model, and broad object-store operations. |
| Apache Ozone | Yes | Strong open-source on-prem object-storage candidate with Ozone Manager/SCM/Datanodes, S3 Gateway, Kerberos/Ranger integration options, Recon, replication, and erasure coding. |

| Candidate | Advanced to detailed fit? | Reason |
|---|---|---|
| OpenStack Swift | No for Phase 1 baseline | Mature object store, but its native API and operational ecosystem are less directly aligned to the Iceberg/Spark/Trino S3 client contract chosen for Stratus Phase 1. It can be revisited if Swift is already an enterprise standard. |
| SeaweedFS | No for production baseline | Useful lightweight distributed file/object system, but not selected as the primary governed lakehouse storage substrate without deeper evidence for Iceberg/Polaris/Spark/Trino production behavior, security, and operations. |
| DAOS | No for Phase 1 baseline | Strong HPC-oriented object storage, but the Stratus Phase 1 contract is S3 lakehouse compatibility and general on-prem platform operations, not a specialized HPC storage interface. |

Only Ceph RGW and Apache Ozone advance to detailed comparison because both plausibly satisfy the Stratus requirement for open-source, production-capable, on-prem object storage with an S3 access path.

#### 2.8.3 Ceph RGW vs Apache Ozone requirements fit

Scoring scale:

- `5` = strong fit with low validation risk
- `4` = good fit with normal release/configuration validation required
- `3` = viable, but material proof-of-fit required before approval
- `2` = weak fit or requires an architectural change
- `1` = poor fit for the stated requirement
- `0` = does not satisfy the requirement

| Requirement | Ceph RGW fit | Ceph score | Apache Ozone fit | Ozone score | Decision implication |
|---|---|---|---|---|---|
| On-prem open-source production storage | Strong. Ceph is a production distributed storage platform with MON/MGR/OSD/RGW services. | 5 | Strong. Ozone is an Apache distributed object store with OM/SCM/Datanodes and production deployment patterns. | 5 | Both remain viable. |
| S3 API coverage for lakehouse engines | Strong baseline. Ceph documents S3-compatible object access and support for core bucket/object/multipart operations. Must still test the exact release. | 5 | Viable but must be proven. Ozone S3 Gateway provides S3 access, but the design must verify the subset needed by Iceberg/Polaris/Spark/Trino. | 3 | Ceph has the lower S3-compatibility risk for the selected client contract. |

| Requirement | Ceph RGW fit | Ceph score | Apache Ozone fit | Ozone score | Decision implication |
|---|---|---|---|---|---|
| Iceberg/Spark/Trino fit | Strong if S3FileIO, Spark S3 client, and Trino endpoint/path-style settings pass against RGW. | 4 | Viable if the selected Ozone S3 Gateway release passes the same Iceberg/Spark/Trino endpoint and path-style tests. | 3 | Ceph has the lower assumed compatibility risk, but both must be proven. |
| Kerberos/Ranger alignment | Possible through adjacent identity/policy integrations, but not the native center of the object-store contract. | 3 | Stronger integration options if Stratus later requires storage-layer Ranger/Kerberos enforcement. | 4 | Ozone gains weight only if storage-layer policy enforcement becomes an explicit requirement. |
| Analytical user authorization model | Storage layer uses service credentials; analytical user policy is enforced later through Polaris/Trino/Ranger. | 4 | Can support stronger storage-layer authorization, but that is not required by the current Phase 1 storage contract. | 4 | Both are acceptable if user-facing authorization stays above storage. |
| Operational model | Strong but complex. Ceph has mature health, dashboard, metrics, CRUSH placement, recovery, and cephadm lifecycle management. | 4 | Strong but also complex. Ozone has OM/SCM HA, Recon, Datanodes, security, and object-store operational patterns. | 4 | Team skill set matters; both require real operators. |
| Performance evidence path | Strong tooling and metrics path through RGW, Ceph Dashboard, Prometheus, and client-side S3 measurements; still must be measured on target hardware. | 4 | Viable through Ozone Recon, S3 Gateway metrics, and client-side S3 measurements; proof-of-fit must validate the selected gateway release. | 3 | Ceph has the lower measurement-risk baseline, but neither candidate is approved without workload evidence. |
| Metadata and small-object behavior | Mature RGW/bucket-index behavior, but Iceberg metadata and small-file stress must be measured explicitly. | 4 | Viable, but S3 Gateway list behavior and metadata-heavy Iceberg layouts need stronger proof before approval. | 3 | Synthetic behavior is proved in Increment 1; real Iceberg |

| Requirement | Ceph RGW fit | Ceph score | Apache Ozone fit | Ozone score | Decision implication |
|---|---|---|---|---|---|
| | | | | | behavior is a required Increment 2/3 and readiness gate, not optional later tuning. |
| Cost/capacity and operator effort | Strong capacity and failure-domain modeling through Ceph pool/CRUSH design; operator skill requirement is material. | 4 | Strong storage-specific model with its own operational complexity; operator skill requirement is material. | 4 | Both require an accepted cost/capacity model and named operating team. |
| Failure-domain and recovery controls | Strong. CRUSH, pools, OSD health, backfill/recovery behavior are central Ceph concepts. | 5 | Strong. Ozone has replication/erasure coding and service HA patterns. | 4 | Both viable; prove via drills. |
| Encryption and key management | Supported through Ceph/RGW encryption options and storage-layer controls; exact mode must be release-validated. | 4 | Supported through Ozone/KMS/transparent data encryption path; exact mode must be release-validated. | 4 | Both viable; neither should be assumed from AWS S3 semantics. |
| Future multi-protocol storage | Strong if Stratus later needs object plus block/file from one storage substrate. | 5 | Strong for object storage; less relevant if Stratus later needs block/file from the same storage substrate. | 3 | Ceph has broader infrastructure-storage optionality. |
| **Total** | | **51 / 60** | | **44 / 60** | Ceph scores higher under the current Stratus requirements because the comparison scores stated storage, S3 compatibility, security, operations, recovery, |

| Requirement | Ceph RGW fit | Ceph score | Apache Ozone fit | Ozone score | Decision implication |
|---|---|---|---|---|---|
| | | | | | performance evidence, metadata behavior, and cost/operator needs. |

The score is not the decision by itself. Both Ceph RGW and Apache Ozone expose S3-compatible APIs, so "S3-compatible" alone does not decide this. Under the current Stratus requirements, the tradeoff is compatibility risk and operating model: Ceph RGW is the selected lower-risk S3 API target for the Iceberg/Spark/Trino/Polaris path. Ozone remains technically plausible but is superseded unless a recorded reconsideration trigger opens a new architecture decision.

#### 2.8.4 Decision due diligence before Increment 1

Ceph RGW is the selected Phase 1 implementation baseline. Before implementation starts, the team records the following decision due-diligence evidence:

- Official release and deployment method selected and pinned.
- release documentation shows the required S3 operations and deployment topology are supported or identifies the exact items Increment 1 must prove
- the target Linux, container runtime, node, disk, network, failure-domain, TLS, monitoring, recovery, and upgrade assumptions are documented
- a preliminary capacity and operator-ownership model is accepted
- the required service-identity boundaries and active Ceph client contract (`CEPH_RGW_ENDPOINT`, scoped RGW credentials, and `S3_PATH_STYLE_ACCESS`) are defined
- measurable Increment 1 and later cross-increment proof targets are recorded with owners and owning gates
- Apache Ozone is retained only as a superseded alternative with explicit reconsideration triggers.

Increment 1 determines whether the selected Ceph implementation passes its engineering gate. A failed storage-only gate stops Increment 2 and triggers remediation or a new architecture decision record. A later engine-specific failure reopens Ceph qualification but does not create a circular prerequisite for Increment 1.

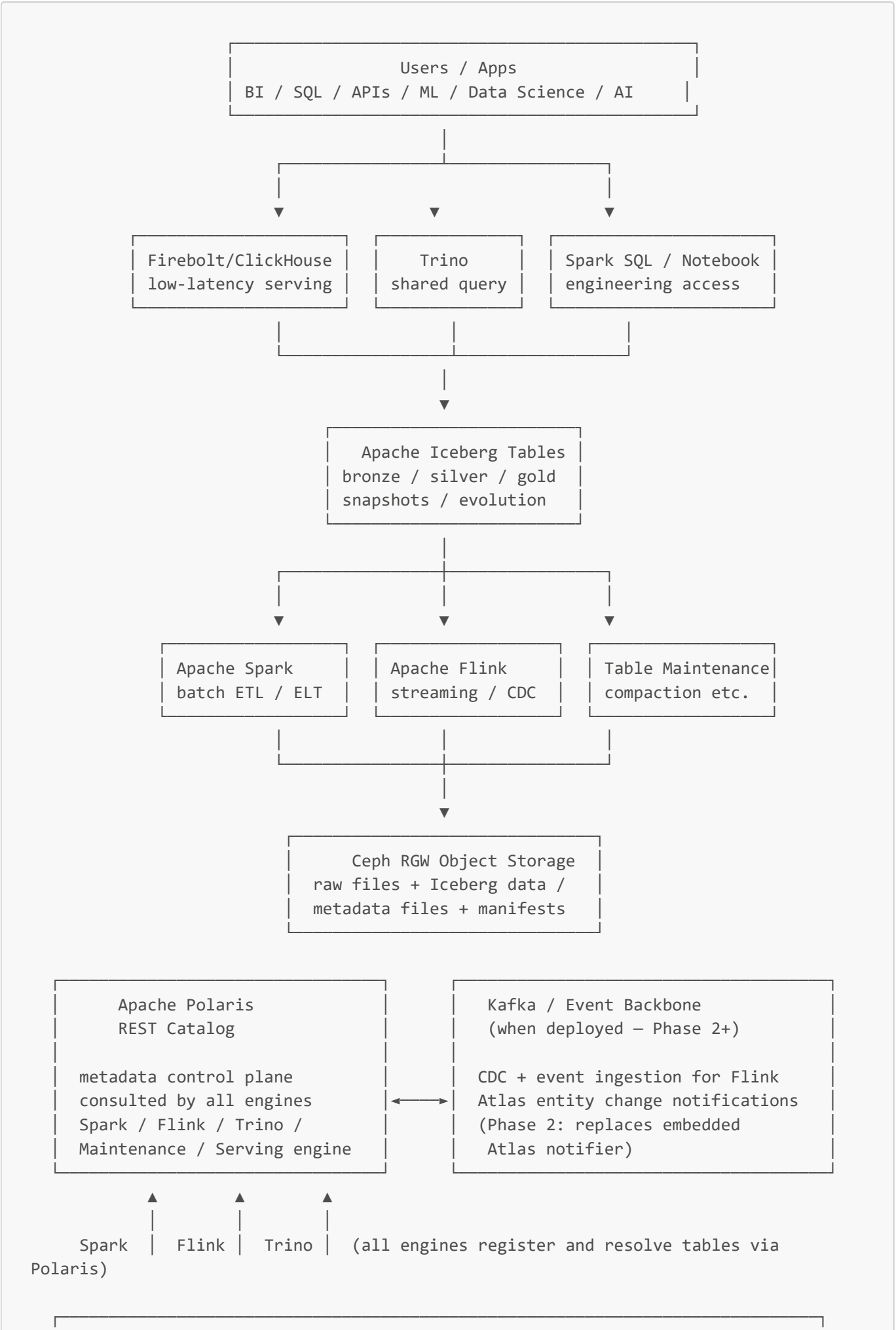
#### 2.8.5 Decision summary

Ceph RGW is the selected Phase 1 baseline because Stratus depends on the exact S3-compatible behavior required by Iceberg, Spark, Trino, Polaris, Airflow, and the Java verification suite. The selection does not waive proof: the developer profile must pass the client contract quickly, and the production profile must additionally pass distributed-storage, failure-domain, recovery, capacity, security, and operations gates.

Apache Ozone remains a documented superseded alternative. It is reconsidered only through a new architecture decision if requirements shift toward storage-layer Ranger/Kerberos enforcement or Ozone's volume/bucket namespace. That change would require revising the storage contract and the downstream Increment 2/3/5 engine configuration, not merely swapping an endpoint URL.

---

## 3. Logical Architecture


Users / Apps
BI / SQL / APIs / ML / Data Science / AI
Firebolt/ClickHouse
low-latency serving
Trino
shared query
Spark SQL / Notebook
engineering access
Apache Iceberg Tables
bronze / silver / gold
snapshots / evolution
Apache Spark
batch ETL / ELT
Apache Flink
streaming / CDC
Table Maintenance
compaction etc.
Ceph RGW Object Storage
raw files + Iceberg data /
metadata files + manifests
Apache Polaris
REST Catalog
metadata control plane
consulted by all engines
Spark / Flink / Trino /
Maintenance / Serving engine
Kafka / Event Backbone
(when deployed — Phase 2+)
CDC + event ingestion for Flink
Atlas entity change notifications
(Phase 2: replaces embedded
Atlas notifier)
Spark | Flink | Trino | (all engines register and resolve tables via Polaris)

```
    │ Governance / Control Plane                                                  │
    │ Apache Atlas — metadata, lineage, glossary; backing services vary by profile│
    │ Apache Ranger — policy enforcement, classification-driven access control    │
    │ Airflow — orchestration, scheduling, promotion gates, maintenance           │
    │ FreeIPA — Kerberos, LDAP, PKI          Keycloak — OIDC for REST services    │
    └─────────────────────────────────────────────────────────────────────────────┘
```

---

# 4. Core Components and Responsibilities

## 4.1 Ceph RGW Object Storage

### Role

Ceph RGW is the durable persistence substrate. It exposes the Ceph storage cluster through an S3-compatible API and stores:

- landed raw files
- parquet/orc data files
- Iceberg metadata files
- manifests and snapshots
- curated datasets
- archive and retention data

### Responsibilities

- high-scale durable storage
- cheap separation of storage from compute
- storage for both raw and curated datasets
- support for Iceberg metadata and data files

### Design Position

Do **not** expose raw files in object storage as the enterprise data contract.

That is a bad pattern because:

- schemas become implicit or tribal knowledge
- partitions become engine-specific guesswork
- concurrency becomes unsafe
- consumers couple themselves to physical file layout

The contract must be **Iceberg tables**, not folders and filenames.

### Bucket and object layout

The foundation uses a small number of platform buckets rather than one bucket per dataset:

| Bucket | Purpose |
|---|---|
| `stratus-landing` | raw source files and bounded external extracts before table ingestion |

| Bucket | Purpose |
|---|---|
| `stratus-bronze` | bronze Iceberg data and metadata |
| `stratus-silver` | silver Iceberg data and metadata |
| `stratus-gold` | gold Iceberg data and metadata |
| `stratus-platform` | platform-internal data such as quality results, Spark event logs, audit extracts, and maintenance metadata |

Landing-zone object keys should be predictable and source-oriented:

```
s3://stratus-landing/<source-system>/<dataset>/<ingest-date>/<file-name>
```

For example:

```
s3://stratus-landing/crm/customers/2026-07-04/customers.csv
```

Landing files are transient inputs to governed table creation. Retention should be long enough to support replay and audit, but consumers should not query landing files directly.

---

## 4.2 Apache Iceberg

### Role

Iceberg is the open table format and the core semantic layer of the platform.

### Why Iceberg is foundational

Apache Iceberg provides capabilities including:

- schema evolution
- partition evolution
- hidden partitioning
- snapshots and time travel
- rollback
- metadata-based planning
- multi-engine interoperability

References:

- Docs: https://iceberg.apache.org/docs/latest/
- Spark quickstart: https://iceberg.apache.org/spark-quickstart/
- Flink integration: https://iceberg.apache.org/docs/latest/flink/

### Design rules

- all bronze, silver, and gold analytical datasets should be Iceberg tables

- all engines should go through the chosen Iceberg catalog strategy
- table naming, namespace, retention, and ownership standards must be enforced centrally
- maintenance operations such as compaction and snapshot expiry must be owned explicitly

### Why this matters

Iceberg prevents the platform from collapsing into a set of loosely-related files. It gives the storage layer transactional table semantics suitable for multiple engines.

---

## 4.3 Apache Spark

### Role

Spark is the batch and large-scale transformation engine.

### Best-fit responsibilities

- heavy ETL / ELT
- backfills
- historical reprocessing
- large joins
- quality standardisation
- enrichment at scale
- feature engineering
- materialising silver and gold tables

### Why Spark belongs here

Spark remains the best fit for:

- large bounded workloads
- historical reshaping
- expensive joins and aggregations
- notebook-based engineering workflows

### Design rules

- Spark writes to and reads from Iceberg tables rather than unmanaged file paths
- Spark jobs should be orchestrated by Airflow for batch workflows
- large maintenance jobs should be isolated from business-critical serving windows
- a Spark Structured Streaming job is a long-running service, not a scheduled task, and is operated as one even though it shares the batch cluster

Spark is also one of three runtimes that can write an event topic into an Iceberg table, alongside Flink and the Iceberg Kafka Connect sink. Because Spark is delivered in the foundation stage while the other two arrive with streaming, micro-batch on the existing Spark cluster reaches minutes-level freshness without introducing a new runtime; §6.4.7 compares the three and states the decision order.

---

## 4.4 Apache Flink

### Role

Flink is the real-time and streaming engine.

### Best-fit responsibilities

- CDC ingestion
- event stream ingestion
- continuous enrichment
- stateful stream processing
- exactly-once stateful computation where needed
- near-real-time writes into Iceberg

### Why Flink belongs here

Flink is built for bounded and unbounded streams and supports stateful stream processing with event-time semantics. Iceberg explicitly documents Flink integration for reading and writing tables:

- Flink integration docs: https://iceberg.apache.org/docs/latest/flink/

### Design rules

- Flink should own real-time data movement and continuous pipelines
- Flink should not be replaced by Airflow for event-driven processing
- streaming write patterns into Iceberg should be tested carefully for commit cadence, compaction, and consumer freshness
- in steady state, Flink should be the sole writer for streaming-owned tables rather than sharing write ownership casually with batch jobs

Flink is one of three runtimes that can write a topic into an Iceberg table, alongside Spark Structured Streaming and the Iceberg Kafka Connect sink. It is selected where freshness is genuinely sub-second or the job needs continuous keyed state; §6.4.7 compares the three and states the decision order.

---

## 4.5 Apache Kafka

### Role

Kafka is the preferred event backbone when the platform requires durable, replayable shared streaming and CDC use cases.

### Responsibilities

- CDC event transport
- business event ingestion
- replayable streams for Flink consumption
- buffering and back-pressure smoothing between producers and stream processors
- Atlas entity change notification bus (Phase 2+)

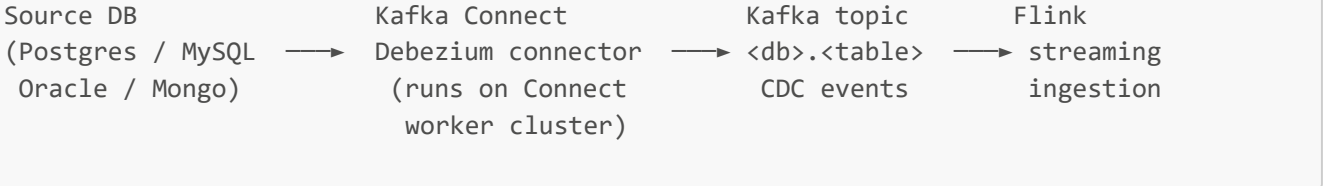
### Kafka Connect and Debezium

**Kafka Connect** is the standard data integration framework bundled with Apache Kafka (Apache 2.0 licence). It runs as a cluster of worker processes alongside the Kafka brokers and manages connectors that move data between Kafka and external systems. No additional product or commercial dependency is introduced.

**Debezium** (Apache 2.0, Red Hat-sponsored) is the CDC connector library that runs inside Kafka Connect. It captures row-level change events directly from source database transaction logs and publishes them as structured Kafka topic messages.

Connect is a connector runtime, not a single-purpose CDC tool. In Stratus it hosts **source** connectors only — Debezium capturing into Kafka topics. The **Apache Iceberg sink connector** would run on this same worker cluster and write topics directly into Iceberg tables, with no stream processor in the path. That capability is deliberately not part of the selected design: it is one of the three writer options compared in §6.4.7, and adopting it would extend this component's responsibilities from ingestion into table writes, with the commit-coordination, duplicate, and failure-posture consequences recorded there.

Together, Kafka Connect + Debezium form the CDC ingestion path:




**Supported source systems**

- PostgreSQL (via pgoutput or wal2json logical replication)
- MySQL / MariaDB (via binlog)
- Oracle (via LogMiner)
- SQL Server (via CDC)
- MongoDB (via change streams)

**Design rules**

- Kafka Connect workers run as a dedicated cluster, co-located with or alongside Kafka brokers
- each source system CDC feed is managed as a named Debezium connector with explicit configuration versioned in source control
- connector configuration, offsets, and status are stored in dedicated Kafka topics — no external state store required
- Flink consumes CDC topic messages directly; no intermediate transformation layer between Debezium output and Flink ingestion jobs
- Connect hosts source connectors only; no sink connector writes to a governed Iceberg table without the comparison in §6.4.7 being worked through first, because that changes which component owns table writes
- connector deployment and lifecycle management is owned by the platform team, not individual domain teams

References:

- Apache Kafka Connect: https://kafka.apache.org/documentation/#connect
- Debezium: https://debezium.io/

- Apache Iceberg Kafka Connect sink, the sink-side use of this same runtime (§6.4.7): https://iceberg.apache.org/docs/latest/kafka-connect/

### When Kafka is justified

- continuous CDC ingestion with replay requirements
- multiple downstream consumers on the same event stream
- bursty producers that need durable buffering
- event-driven platform patterns beyond analytical table production

### When Kafka is not required in the foundation

- scheduled batch ingestion dominates the workload
- source systems land files or bounded extracts in object storage
- Spark-driven bounded processing is the primary operating model
- near-real-time needs are modest and do not require a shared replayable event log

### Design Position

Kafka should not be treated as automatically mandatory just because Flink exists in the architecture. The **shared platform event backbone** becomes a Phase 2 core component when the platform needs durable event retention, independent consumers, replay, back-pressure absorption, or CDC at meaningful scale. Phase 1 does not deliver those streaming capabilities. A small external Kafka service brought forward solely as Atlas's supported notification dependency remains part of the Atlas production topology, not a completed Kafka platform increment.

Kafka is delivered by Increment 8. Kafka Connect and Debezium are delivered by Increment 9 and are required before the CDC path is accepted; they are separate increments so the event backbone can be secured and proven first.

## 4.5.1 Qualified alternative: Apache Pulsar

Kafka is the selected backbone. **Apache Pulsar is a qualified alternative**, evaluated and recorded in ADR-P1-005 rather than dismissed, because three of its properties fit the Stratus deployment context specifically. This subsection exists so that a future reconsideration starts from evidence rather than from a fresh product comparison.

**What Pulsar would gain**

- **Serving and storage scale separately.** Brokers are stateless and own no data; Apache BookKeeper bookies own durable replicated storage. Adding consumers or topics does not force storage expansion, and broker rebalancing moves no data. On fixed on-premises hardware that decouples two capacity decisions Kafka makes together.
- **Tiered storage onto the object store already deployed.** Sealed segments offload to S3-compatible storage while the active segment stays in BookKeeper, so the event log's cold backlog could live in the same Ceph RGW cluster as the lakehouse instead of on broker-local disk sized for peak retention.
- **Native multi-tenancy.** Tenants and namespaces carry quotas, retention, and authorization, making domain isolation a configuration concern rather than a cluster-per-domain decision.
- **Per-key order under parallel consumption.** Key_Shared subscriptions preserve order per key while multiple consumers process one topic, which is the property the CDC merge rule in §6.4.4 depends on.

**What adopting it would cost**

- **Three moving parts in production** — brokers, bookies, and a metadata store — against Kafka's single broker role in KRaft mode. Bookie replacement, ledger recovery, and metadata-store availability each become first-class runbooks.
- **Narrower packaged CDC coverage.** Pulsar IO packages Debezium connectors for MySQL, PostgreSQL, and MongoDB only. Oracle and SQL Server capture requires the separate Debezium Server runtime with its Pulsar sink, so a platform ingesting from those sources operates two CDC runtimes where Kafka Connect runs all five on one framework.
- **Smaller operational talent pool and fewer worked on-premises examples**, so runbooks must be written rather than adapted.
- **Metadata-store churn upstream.** ZooKeeper remains supported, Oxia is recommended for new clusters from Pulsar 5.0, and the etcd backend is removed in 5.0.

**What it would not change**

Apache Atlas publishes and consumes entity change notifications over Kafka topics (`ATLAS_HOOK` inbound, `ATLAS_ENTITIES` outbound) as a property of the product. Verified against Atlas sources on 2026-08-06: the notification factory selects between exactly two implementations, `KafkaNotification` and `RestNotification`, rather than exposing a pluggable provider, and the REST path (`atlas.hook.rest.notification.enabled`) only changes how *hooks* reach Atlas — the server publishes to Kafka behind that endpoint and the outbound stream is unchanged. **Adopting Pulsar would therefore add a messaging system rather than replace one**, since the Atlas notification service would still be Kafka. That single fact is the strongest argument for the current selection: with Kafka as the backbone, Atlas consolidates onto it as planned in Increment 12; with Pulsar, the platform operates both permanently.

**When to reconsider**

Revisit this decision if event retention grows large enough that offloading the backlog to Ceph RGW is materially cheaper than provisioning broker storage, if domain isolation requirements outgrow topic-prefix conventions, or if Atlas gains support for a second notification transport. Reconsideration before Increment 8 is cheap; after it, the migration cost is real.

References:

- Apache Pulsar: https://pulsar.apache.org/
- Pulsar architecture overview: https://pulsar.apache.org/docs/concepts-architecture-overview/
- Pulsar tiered storage: https://pulsar.apache.org/docs/tiered-storage-overview/
- Pulsar IO Debezium connectors: https://pulsar.apache.org/docs/io-cdc-debezium/
- Debezium Server, including the Pulsar sink: https://debezium.io/documentation/reference/stable/operations/debezium-server.html

## 4.6 Apache Atlas

### Role

Atlas is the metadata and governance plane.

### Responsibilities

- technical metadata catalog
- business glossary
- lineage

- data classifications and tags
- ownership and stewardship metadata
- discovery and search

Reference:

- Apache Atlas project: https://atlas.apache.org/

## Operational dependencies

Atlas requires graph, search, and notification services. Stratus uses the minimal topology only for developer work and requires supported external dependencies before Atlas is accepted for production:

| Dependency | Developer profile | Production profile |
|---|---|---|
| Graph store | embedded/disposable backend supplied by the approved Atlas development distribution | external supported HBase topology with durable storage, backup, monitoring, and tested recovery |
| Search index | embedded/disposable Solr | external SolrCloud with ZooKeeper, persistent collections, backup, monitoring, and tested recovery |
| Notification bus | embedded notifier for local functional testing only | external supported Kafka notification service; production may bring the Phase 2 Kafka backbone forward as a dependency without enabling CDC/Flink, or use an Atlas-dedicated cluster that Increment 12 later consolidates |

The developer profile may run Atlas as one disposable service. It is not eligible for the Phase 1 production-readiness gate. Production acceptance requires the external dependency topology, two Atlas application instances or an approved availability exception, encrypted service protocols, and restore evidence for graph, search, types, glossary, and classifications. If the production notification service is deferred, Atlas production acceptance and governed production dataset onboarding are also deferred; the lab may continue.

The production dependency choice is validated against the selected Atlas release documentation. Embedded stores are convenience tooling, not a small-production sizing tier.

## Blunt reality

Atlas is useful, but it is not plug-and-play magic. It often becomes the most integration-heavy element in the stack.

The hard part is not installing Atlas. The hard part is:

- harvesting schemas and table metadata reliably
- pushing lineage from Spark and Flink jobs
- mapping technical assets to business terms
- keeping metadata current and trusted

## Design rules

- treat metadata publication as part of pipeline completion, not an optional afterthought
- define mandatory metadata fields for every dataset

- define ownership, data steward, SLA, sensitivity, and domain tags
- integrate Atlas with data quality and promotion workflows where possible
- treat lineage emission from Spark and Flink jobs as a delivery contract, not an aspiration

---

## 4.7 Apache Ranger

### Role

Ranger is the authorization and policy enforcement plane for data access controls.

### Responsibilities

- enforce role-based access by domain and environment
- apply tag-based or classification-driven policies for sensitive datasets
- align policy enforcement with Atlas classifications where possible
- provide an auditable enforcement path instead of relying on naming conventions and process discipline alone

### Design Position

Atlas without an enforcement layer is incomplete for sensitive-data governance. The platform should pair Atlas metadata and classifications with Ranger policy enforcement so classifications can drive real access rules.

---

## 4.8 Apache Airflow

### Role

Airflow is the workflow orchestration and control-plane scheduler.

### Best-fit responsibilities

- Spark job scheduling
- bounded batch workflow orchestration
- dependency management
- backfills
- table maintenance jobs
- data quality jobs
- metadata sync workflows
- promotion gates from bronze to silver to gold

Reference:

- Apache Airflow overview: https://airflow.apache.org/

### What Airflow should not do

Airflow should not be treated as:

- a streaming runtime
- a low-latency event processor
- a substitute for Flink

- an always-on stateful compute system

### Design rules

- use Airflow for finite workflows and control-plane operations
- keep DAGs readable and domain-oriented
- avoid building all platform logic into Airflow itself
- use Airflow to orchestrate engines, not replace them
- keep transformation, quality, and maintenance logic in versioned Spark jobs or platform libraries
- make promotion gates explicit tasks that fail closed
- include a stable `run_id` in every DAG run, Spark job, quality record, and lineage event
- treat DAG import errors as platform incidents, not harmless UI noise

### Deployment posture

The developer profile uses Docker Compose with Airflow 3.3.1, one API server, one DAG processor, one scheduler, one triggerer, PostgreSQL for metadata, and `LocalExecutor`. This is sufficient for functional development evidence and keeps the local operational surface small; it is not a production topology. Production uses the same accepted image, DAGs and contracts but requires durable external metadata and logs, managed secrets, trusted identity, backup/restore, availability and capacity evidence. If concurrency, isolation or placement becomes a measured constraint, move to CeleryExecutor or KubernetesExecutor as a deliberate scaling step.

Where Podman is selected for production Linux hosts, services use Quadlet or another current approved unit mechanism. The deprecated `podman generate systemd` workflow is excluded.

Airflow's metadata database is part of the control plane and must be backed up. Losing it means losing run history, task state, retry state, and operational audit context.

Airflow image assembly and Airflow deployment are separate lifecycle stages. The control-plane layer is based on the pinned official Airflow image; the Spark submission layer is sourced from a pinned official Spark Java 21 image through OCI registry layers. Spark and PySpark distributions are not copied as duplicate gigabyte-scale artifacts through a developer workstation build context. The result is built, scanned, published, and recorded once, then consumed by immutable digest during deployment.

Developer lifecycle tests never build or scan images. Repository guardrails, candidate-image smoke tests, Airflow lifecycle tests, real Spark submission, and release security assessment are separate gates with phase timings. Live tests reuse a suite-scoped stack where isolation allows and use unique run identifiers instead of repeatedly starting the data plane for tiny assertions. The dated decision and measured baseline are recorded in `airflow_spark_runtime_reassessment_20260818.md`.

---

## 4.9 Trino

### Role

Trino is the default open interactive SQL and shared query plane over governed Iceberg datasets.

### Best-fit responsibilities

- analyst and BI access to curated datasets
- shared SQL access across domains
- ad hoc query and federated read patterns where appropriate

- open query access when no acceleration engine is deployed

### Design position

Spark SQL and notebooks remain engineering tools. They should not be treated as the default enterprise interactive query plane.

### Design rules

- Trino must use the central Apache Polaris REST catalog for Iceberg table discovery.
- Trino must query Iceberg tables, not raw object-storage paths.
- Trino is the shared read/query plane, not the primary ETL engine for bronze-to-silver or silver-to-gold processing.
- Trino query validation should compare row counts, aggregates, schemas, and quality-result visibility against Spark-produced outputs.
- Trino access should initially be constrained to internal platform validation, then integrated with Ranger and Keycloak/FreeIPA as the governance and identity increments land.
- Trino must expose `platform.quality_check_results` so operators and analysts can inspect quality outcomes without Spark access.

### Query contract

Increment 5 should prove that Trino can answer the same business questions as Spark over the same Iceberg snapshots. The minimum query contract is:

| Query class | Expected behavior |
|---|---|
| Discovery | `SHOW SCHEMAS` and `SHOW TABLES` expose Polaris namespaces and tables |
| Bronze validation | raw row counts match Spark ingestion output |
| Silver validation | deduplicated row counts and schema match Spark transform output |
| Gold validation | aggregate results match Spark materialisation output |
| Quality visibility | `stratus.platform.quality_check_results` is queryable |
| Error behavior | invalid columns and missing tables fail with clear SQL errors |
| Cross-zone query | joins across bronze, silver, and gold work where policy permits |

---

## 4.10 Query Acceleration Layer — Firebolt Core or ClickHouse

### Role

The optional query acceleration layer is a high-performance serving engine for interactive SQL over curated Iceberg datasets. Two candidate engines are carried in this architecture:

- **Firebolt Core** — high-performance serving engine for interactive SQL over external data and Iceberg datasets
- **ClickHouse** — open-source columnar OLAP database with low-latency, high-concurrency analytics and Iceberg table integration

References:

- Firebolt Iceberg and external data: https://docs.firebolt.io/performance-and-observability/iceberg-and-external-data
- ClickHouse documentation: https://clickhouse.com/docs
- ClickHouse Iceberg integration: https://clickhouse.com/docs/en/engines/table-engines/integrations/iceberg

### Best-fit responsibilities

- low-latency BI queries
- interactive dashboards
- app-facing analytics
- serving curated analytical datasets
- query acceleration over Iceberg-backed data

### Design position

Whichever engine is selected, the acceleration layer sits **northbound of Iceberg** and serves curated data products. It must not become the foundational metadata or ingestion layer.

### When it fits well

- demanding dashboard latency requirements
- high-concurrency SQL serving
- curated data marts and semantic consumption layers

### When to be cautious

- when the platform has not yet stabilised its Iceberg and governance foundations
- when cost or licensing complexity is unclear
- when the operating model is already too heavy
- when the platform has not yet proven a stable curated layer and a real low-latency concurrency requirement

> **TODO:** The engine choice between **Firebolt Core** and **ClickHouse** is deliberately open. The Phase 3 acceleration evaluation (Increment 17) must compare the two on Iceberg read support, governance fit, and operating cost before the platform commits to a serving engine.

---

## 5. Data Quality Subsystem

The data quality subsystem is built entirely from components already in the platform stack. No additional framework is introduced.

### 5.1 Design position

Quality execution, result storage, orchestration, metadata publication, and access control are all handled by existing platform components:

- **Spark** executes quality checks as bounded Spark jobs using DataFrame assertions and SQL
- **Iceberg** stores quality results as a first-class platform table, queryable and versioned like any other dataset
- **Airflow** orchestrates check execution and enforces promotion gates as explicit DAG tasks
- **Atlas** carries quality status as a metadata attribute on each dataset, updated on every check run
- **Ranger** can restrict access to datasets that have not passed promotion gates
- **Trino** provides ad-hoc SQL access to quality results for operators and analysts

## 5.2 Quality check execution

Quality checks run as dedicated Spark jobs. Each job targets a specific dataset and zone transition — for example, bronze validation before silver promotion, or silver validation before gold materialisation.

### Check types

| Check type | Description |
|---|---|
| Schema conformance | column names, types, and nullability match the registered schema |
| Completeness | row count meets minimum threshold; mandatory columns have no null rate above tolerance |
| Uniqueness | primary or business key columns contain no unexpected duplicates |
| Freshness | latest record timestamp is within the expected SLA window |
| Referential integrity | foreign key values resolve against reference datasets |
| Business rule | domain-specific assertions expressed as SQL predicates or DataFrame conditions |

### Check definition

Checks are defined as configuration — YAML or equivalent — co-located with the pipeline definition and version-controlled alongside job code. Each check definition specifies:

- target dataset (namespace and table name in Polaris)
- check type and parameters
- pass threshold (absolute or percentage tolerance)
- severity: `blocking` or `warning`
- owner

Blocking checks must pass before promotion proceeds. Warning checks are recorded but do not halt the pipeline.

## 5.3 Quality result storage

Every check run writes a result record to a dedicated platform Iceberg table:

```
platform.quality_check_results
```

### Result record schema

| Column | Type | Description |
|---|---|---|
| run_id | string | unique identifier for the check run |
| dataset_namespace | string | Polaris namespace |
| dataset_name | string | Iceberg table name |

| Column | Type | Description |
|---|---|---|
| zone | string | bronze / silver / gold |
| check_type | string | completeness / uniqueness / freshness / etc. |
| check_name | string | descriptive name of the specific check |
| severity | string | blocking / warning |
| status | string | passed / failed / warning |
| metric_value | double | observed metric (e.g. null rate, row count) |
| threshold | double | configured pass threshold |
| failure_detail | string | human-readable failure context |
| pipeline_run_id | string | Airflow DAG run ID |
| checked_at | timestamp | check execution time |
| iceberg_snapshot_id | long | Iceberg snapshot ID of the checked dataset |

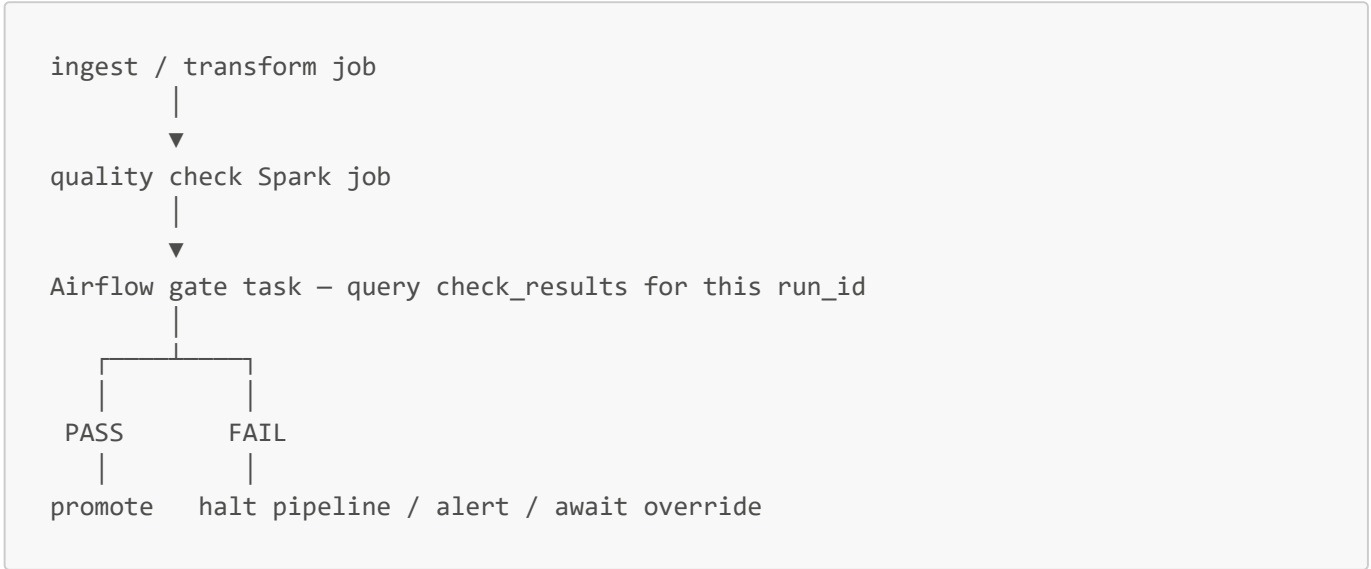
Results are append-only. Historical quality records are retained as a permanent audit trail. Trino and Spark SQL can query this table directly.

---

## 5.4 Promotion gates

Promotion from one zone to the next is an explicit Airflow task, not an implicit side effect of a transformation job.

### Gate pattern




The Airflow gate task queries `platform.quality_check_results` for the exact quality `run_id` and source table, asserts that all blocking checks have status `passed`, and only then triggers the downstream transformation or materialisation task. The writer rechecks the same evidence immediately before writing as defence in depth; the explicit Airflow task remains the visible orchestration and audit boundary. See ADR-P1-007.

### Override model

A failed blocking check halts the pipeline. Manual override requires:

- an explicit Airflow task approval by a named data steward or platform operator
- an override reason recorded in the pipeline run metadata
- the override event written to `platform.quality_check_results` as a separate audit record with status `overridden`

Overrides are auditable. They do not silently suppress the failure record.

---

### 5.5 Atlas metadata integration

On completion of each check run, the pipeline publishes quality status back to Atlas for the target dataset:

- `quality_status`: `passed` / `failed` / `warning`
- `quality_last_checked`: timestamp of the most recent check run
- `quality_run_id`: link to the check run in `platform.quality_check_results`
- `quality_blocking_failures`: count of blocking check failures in the last run

This means Atlas dataset entries reflect current quality state and consumers can discover whether a dataset is in a passing or failing quality condition without querying the results table directly.

Quality status publication to Atlas is a mandatory step in every pipeline's completion contract, not an optional integration.

---

### 5.6 Access control integration

Ranger policies can be applied to restrict read access to datasets in a failing quality state where the sensitivity of the data and the nature of the failure warrant it. This is a platform-level policy decision, not automatic — classification-driven Ranger rules should be applied deliberately for sensitive zones and datasets.

---

### 5.7 Ownership

- **Platform team** owns the `platform.quality_check_results` table, its schema, retention policy, and Iceberg maintenance schedule
- **Domain pipeline owners** define and maintain check configurations for their datasets
- **Data stewards** hold override authority for their domain's blocking failures
- **No check run should complete without a result record** — silent quality execution is not permitted

---

## 6. Data Lifecycle Model

A three-layer model is recommended.

### 6.1 Bronze

Raw or lightly normalised data.

#### Characteristics

- closest possible fidelity to source
- append-biased

- minimal correction
- schema capture and source metadata preserved
- suitable for replay and audit

### Typical producers

- landed batch files
- CDC feeds
- Flink ingestion pipelines
- external source extracts

### Capture patterns at the source

How data is captured determines what the bronze table can and cannot be trusted to represent. Each pattern is named explicitly per source, because their correctness properties differ.

| Pattern | Mechanism | Correctness limits |
|---|---|---|
| Full snapshot extract | bounded `SELECT` over partitioned key ranges, or a native unload | correct by construction; cost grows with table size, so it suits reference and dimension data |
| Watermark / incremental extract | pull rows where `updated_at` exceeds the last high-water mark | cheapest and most common, but **cannot observe hard deletes**, and breaks on clock skew, transactions committing out of order, and backdated updates; overlap the window and deduplicate downstream |
| File drop / landing zone | vendor delivers to a bucket prefix or transfer location | build the event-driven variant, triggered by object-creation notification; polling a prefix listing degrades badly once it holds large object counts |
| Log-based CDC | read the source transaction log (§4.5) | captures deletes and true transaction order; emits before/after images with a sequence number, which is what makes the downstream merge idempotent and order-safe |
| Bootstrap snapshot plus log tail | chunked incremental snapshot alongside log capture | the supported migration from batch extraction to CDC; Debezium's signal-driven incremental snapshot avoids the table locks and extended copy window of a stop-the-world initial load |

Where an application team proposes writing business state to a database and separately publishing an event, prefer the **outbox pattern**: the service writes state and an event row in one local transaction, and CDC on the outbox table publishes it. Dual writes drift; an outbox cannot.

## 6.2 Silver

Conformed, validated, reusable enterprise data.

### Characteristics

- deduplicated
- typed and normalised
- standard business keys

- reference-data-enriched
- suitable for broad reuse across teams

### Typical producers

- Spark transformation jobs
- Flink continuous enrichment pipelines

## 6.3 Gold

Consumption-ready data products.

### Characteristics

- business-facing marts
- KPI tables
- aggregates
- semantic views
- application-ready analytical tables

### Typical consumers

- BI dashboards
- data APIs
- serving/query engines such as Firebolt Core or ClickHouse
- data science and analytics consumers

### Rule

All three zones should be implemented as **Iceberg tables**, not folder conventions alone.

### Write ownership rule

For any given table, steady-state write ownership should be explicit and narrow. The platform should prefer a one-writer pattern per table where possible, with compaction ownership assigned deliberately and not left ambiguous across engines.

### Dataset naming convention

Dataset names should be stable, lowercase, and domain-oriented:

```
<catalog>.<zone>.<domain>_<dataset>
```

Examples:

```
stratus.bronze.crm_customers
stratus.silver.crm_customers
stratus.gold.sales_customer_summary
```

Small verification tables may use simple names such as `verification_customers`, but production datasets should include a domain prefix. Naming should make ownership and lifecycle obvious without encoding sensitive classifications into names.

### Schema and partition governance

Schema evolution is allowed, but it must be governed:

- additive nullable columns are normally safe
- type widening must be reviewed
- dropping or renaming columns requires compatibility analysis and consumer notification
- partition evolution must be justified by query patterns, not guessed up front
- every table should have an owner-approved retention and maintenance policy before production use

Hidden partitioning should be preferred over exposing physical partition assumptions to consumers. Consumers should query tables, not construct paths.

---

## 6.4 Write modes into Iceberg

§2.3 separates streaming and batch as **compute** concerns — which engine runs the work. This section defines them as **storage** concerns: how data actually lands in an Iceberg table.

The two lanes converge. Whatever the engine, every write ends at the same operation: write Parquet data files into object storage, then atomically commit a manifest that makes them visible. Streaming and batch differ in **commit cadence** and **merge strategy**, not in mechanism. Engine choice does not imply write mode: Spark can write continuously in micro-batch, and Flink can append or upsert. Write mode is therefore a per-table decision with a named owner, not a consequence of which team wrote the job.

#### 6.4.1 The four write modes

| Mode | What the writer produces | Read cost | Write cost |
|---|---|---|---|
| Append-only | new data files only | lowest | lowest |
| Upsert, copy-on-write (CoW) | rewritten data files containing the affected rows | lowest | highest |
| Upsert, merge-on-read (MoR) | new data files plus delete files | grows until compaction | low |
| Buffer-then-merge | continuous append to a staging table, periodic MERGE into the curated table | lowest on curated | amortised |

#### 6.4.2 Scaling and blocking behaviour

Each mode fails in a characteristic way. The failure signature matters more than the theory, because it is what an operator will actually observe.

**Append-only**

- *Pros*: cheapest commit; no read or write amplification; concurrent writers rarely contend because they produce disjoint files; fully replayable.
- *Cons*: no in-place correction — restatements must be modelled as new rows carrying a sequence, with deduplication downstream.
- *Scales*: near-linearly with volume and writer count. This is the default and the mode to prefer wherever the data model allows it.
- *Blocks when*: commit cadence is high across many partitions. A 30-second commit interval across 200 partitions produces roughly 576,000 objects per day.
- *Failure signature*: query planning time grows faster than data volume; manifest and metadata file counts climb; object listing slows.
- *Mitigation*: lengthen the commit interval to minutes; schedule compaction as a first-class pipeline stage (§10.3), not as cleanup.

**Upsert, copy-on-write**

- *Pros*: fastest reads — consumers never merge delete files; simplest consumer semantics; the Iceberg default for update, delete, and merge commands.
- *Cons*: write amplification. Updating a single row rewrites the entire data file that contains it — up to `write.target-file-size-bytes`, 512 MB by default.
- *Scales*: with low change rates and read-heavy access — periodically rebuilt silver and gold tables.
- *Blocks when*: change rate multiplied by file size exceeds the write budget. A small daily percentage of changed rows scattered across large files can rewrite most of a table to update very little of it. Concurrent writers touching overlapping files conflict under optimistic concurrency, retry, and can livelock.
- *Failure signature*: job runtime dominated by rewriting unchanged rows; repeated commit retries on the same table.
- *Mitigation*: switch high-churn tables to merge-on-read; partition writers by key range so they touch disjoint files; serialise commits per table.

**Upsert, merge-on-read**

- *Pros*: cheap writes — the writer records deletes instead of rewriting data. This is the only mode that sustains continuous CDC at rate.
- *Cons*: read amplification — readers reconcile delete files against data files. Equality deletes cost more at read time than position deletes. Correctness of performance depends entirely on compaction keeping up.
- *Scales*: with high update rates and continuous ingestion.
- *Blocks when*: compaction falls behind. Delete files accumulate without bound and read latency degrades continuously. This is the characteristic merge-on-read failure: **writes stay fast while reads collapse**, so the problem surfaces to consumers first and to the writing team last.
- *Failure signature*: rising delete-file count in table metadata — already a mandatory maintenance policy field (§10.3) — with read latency degrading while write latency stays flat.
- *Mitigation*: aggressive scheduled compaction with an alert on the delete-file threshold; prefer position deletes where the engine offers the choice.

**Buffer-then-merge**

- *Pros*: decouples ingest latency from merge cost. Ingestion is append-only, so it scales like append; the expensive merge runs on a cadence the platform controls, amortising it across many changes instead of paying it per micro-batch.
- *Cons*: two objects to operate and reason about; the curated table lags staging by the merge interval; staging consumes additional storage.

- *Scales*: best overall shape for CDC into curated tables, and the recommended default when a source produces a continuous change stream that consumers read as current state.
- *Blocks when*: the merge interval is shorter than the merge duration — merges overlap and queue — or merges fail silently and staging grows without bound.
- *Failure signature*: merge duration trending toward the interval; staging row count or lag trending up.
- *Mitigation*: monitor merge duration against its interval as an explicit signal; bound staging retention; alert on curated-table lag.

#### 6.4.3 Choosing a lane

Latency requirement is the deciding factor, and it is routinely overstated. Choosing a lane costlier than the requirement buys nothing but operational load and compaction expense.

| Freshness requirement | Lane | Engine | Typical write mode | Operational cost |
|---|---|---|---|---|
| Sub-second | continuous streaming | Flink over the event backbone (Phase 2+) | append for bronze, merge-on-read for updatable tables | highest |
| Seconds to minutes | micro-batch | Spark Structured Streaming on a 1–5 minute trigger | append, or buffer-then-merge | moderate |
| Hours | scheduled batch | Spark under Airflow | append, or copy-on-write upsert | lowest to operate, highest peak storage load |

Each lane still leaves a choice of writer: Flink, Spark Structured Streaming, and the Iceberg Kafka Connect sink can all commit to the same tables, and the sink sits at micro-batch freshness rather than continuous. §6.4.7 compares the three.

**Batch is cheapest to operate but not cheapest for the storage tier**, and on fixed hardware the distinction matters. A scheduled run moves the same volume and produces a comparable object count, but concentrates it into one window instead of spreading it across the day, so the appliance must be sized for that peak. Elastic cloud storage absorbs the difference; an installed cluster does not. Where the source is already an event topic, batch ingestion is simply the writer tier of §6.4.7 running with a longer window — the buffering, rotation, and commit protocol are identical and only the interval differs — so the lane can be chosen after the writer exists, and revised later without redesign. What batch does **not** fix is a per-message write path: moving one-request-per-message to an overnight schedule leaves the request count unchanged and compresses it into a shorter window, which is worse.

Two reasons legitimately force the continuous lane: a genuine sub-second requirement, or a source that only exposes a log — event topics with no queryable state behind them — in which case there is no batch option to begin with. Absent both, the micro-batch lane satisfies most stated "streaming" requirements at a fraction of the cost, and it does so on the Phase 1 Spark increment without waiting for the Phase 2 event backbone.

#### 6.4.4 Correctness rules

These are mandatory for any table fed by an upsert or CDC path.

1. **Merge on a sequence, not only on a key.** The merge condition must compare a monotonic sequence — log sequence number, SCN, or partition offset — and reject older values. Matching on the business key alone lets a late-arriving older event overwrite newer state. This is the most common silent corruption in CDC pipelines.
2. **Exactly-once is a commit property, not a transport property.** It is achieved by at-least-once delivery plus an idempotent atomic commit keyed on a checkpoint or offset identifier recorded in the Iceberg snapshot metadata. Pursuing exactly-once in the transport layer adds cost without delivering the guarantee.
3. **Commit offsets last.** Source offsets are committed only after the data is durably written *and* the catalog commit has succeeded. Committing on read means a storage failure silently discards a window, with no error and no way to detect the loss afterwards. This ordering is the single rule that hand-built writers most often get wrong; the supported runtimes in §6.4.7 implement it, which is a reason to prefer them over custom code.
4. **Watermark extraction cannot observe hard deletes.** A table fed by `updated_at > last_watermark` extraction silently retains rows deleted at source. Either accept and document that, or capture the source log (§6.1).
5. **Event time and processing time are distinct fields.** Both are recorded; tables partition on event time where consumers filter on it. Watermark windows must overlap and the target must deduplicate, because commits arriving out of order and backdated updates are normal, not exceptional.
6. **One writer per table in steady state**, as required by the write-ownership rule above. Where a second writer is unavoidable, partition writers by key range so they never contend for the same files.

#### 6.4.5 Commit cadence and the small-file tax

Small files are the dominant operational tax on this architecture, and commit cadence is the control. Every table therefore declares a commit cadence alongside the maintenance policy fields in §10.3.

Iceberg targets 512 MB data files by default (`write.target-file-size-bytes` = 536870912). Writers producing files far below that target are trading a small latency gain for a permanent planning cost.

**The governing figure is objects created per second, not bytes stored.** Streaming and batch move the same volume of data; they differ in how many objects that volume is divided into, and that division is a buffering setting rather than a property of either approach. Capacity is added by installing drives. Request and metadata capacity is fixed by the gateway nodes and index hardware already installed, so it is the figure that binds first.

On premises there is no elasticity to absorb the difference, and one logical PUT is not one write:

```
client PUT → RGW gateway → bucket index update → erasure split (k+m) → k+m drive
operations
```

Under an 8+3 erasure profile each object costs eleven fragment writes plus a metadata update. At four objects per second that is negligible; at four hundred it is roughly 4,400 sustained small random writes competing with query traffic and compaction.

Four failure modes follow, in the order they are likely to appear:

| Failure mode | Signature |
|---|---|
| Metadata layer saturation | RGW holds each object in a sharded bucket index in RocksDB omap; plan roughly 100,000 objects per shard. Beyond that, resharding and RocksDB compaction produce latency |

| Failure mode | Signature |
|---|---|
| | spikes indistinguishable from storage saturation |
| Write amplification | as above — the k+m multiplier applies to every object, so object count multiplies drive operations |
| Burst replay | a writer restarting after an outage drains its backlog at maximum rate into a cluster with no headroom, worst of all while a drive is rebuilding |
| Read-side degradation | a partition holding hundreds of thousands of small files spends its query planning time on footer reads and listings |

Design rules that follow:

- Rotate on **5–15 minutes**, not 30–60 seconds. The objective is reducing object count, not chasing freshness the tier is not meant to serve.
- Where sub-minute freshness is genuinely required, serve it from the event backbone — Flink state or a query service reading the topic — and treat the lakehouse as the minutes-latency tier.
- Bound bucket growth structurally, and **pre-shard the bucket index at creation** (`rgw_override_bucket_index_max_shards`) against the object-per-shard budget rather than relying on dynamic resharding, which is disabled on high-ingest buckets and performed deliberately instead. Place the index pool on NVMe.
- Rate-limit the writer so replay cannot saturate the cluster, and back off on 503 responses.
- Budget compaction as consuming the same cluster it is protecting: it reads every small file and writes larger ones back while ingest continues. Schedule off-peak and throttle it.
- Monitor **two** figures against **two** ceilings: requests per second against the gateway limit, and total objects per bucket against the index shard budget. Batching improves the first and does nothing for the second, which only ever grows — which is why snapshot expiry and compaction (§10.3) remain mandatory regardless of write cadence.

Sizing figures must be measured on the installed hardware rather than taken from public-cloud limits: benchmark PUT operations per second until p99 latency rises or 503s appear, repeat against a bucket pre-loaded to a realistic object count to expose metadata degradation, and set the operating budget at half the measured figure, reserving the rest for compaction, replay, and erasure recovery. The minimum rotation interval then follows from `partitions ÷ operating budget`. The detailed measurement and replay-drill procedure is in kafka_to_onprem_lakehouse_design_notes.md.

#### 6.4.6 Zone defaults and table configuration

| Zone | Default write mode | Rationale |
|---|---|---|
| Bronze | append-only, always | source fidelity, replay, and audit require immutability; corrections arrive as new rows, never as mutations |
| Silver | upsert — copy-on-write when change rates are low, merge-on-read when fed continuously | conformed current state; buffer-then-merge is the preferred shape for a CDC source |

| Zone | Default write mode | Rationale |
|---|---|---|
| Gold | copy-on-write upsert, or full rebuild | read-optimised and consumer-facing; write cost is acceptable in exchange for read simplicity |
| `platform.quality_check_results` | append-only | permanent audit trail (§5.3) |

Bronze has one named exception to append-only, recorded in ADR-P1-006: a delivery that arrives a second time may be replaced in place, scoped to that batch alone and opt-in per invocation. The default remains a refusal, so the exception cannot be taken by accident, and `stratus.append-only=true` stays on the table as the marker of the contract Iceberg does not itself enforce.

Write mode is set explicitly per table rather than left to engine defaults:

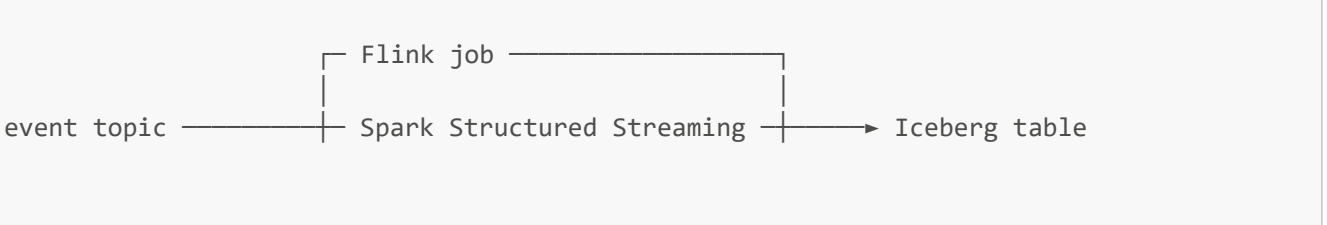
| Property | Values | Iceberg default | Set when |
|---|---|---|---|
| `write.delete.mode`, `write.update.mode`, `write.merge.mode` | `copy-on-write`, `merge-on-read` (format v2 and above) | `copy-on-write` | select `merge-on-read` for continuously updated tables |
| `write.target-file-size-bytes` | bytes | `536870912` (512 MB) | lower only with measured evidence |
| `write.format.default` | `parquet` | `parquet` | retain |
| `write.delete.isolation-level` | `serializable`, `snapshot` | `serializable` | relax only with a recorded justification |
| `write.upsert.enabled` (Flink) | `true`, `false` | not set | Flink upsert writes; requires format v2 and identifier fields, requires partition source columns among the equality fields, and is mutually exclusive with overwrite |

References:

- Iceberg table configuration: https://iceberg.apache.org/docs/latest/configuration/
- Iceberg Flink writes, including upsert requirements: https://iceberg.apache.org/docs/latest/flink-writes/

#### 6.4.7 Choosing the writer

Three runtimes can take a topic and produce Iceberg table commits. They occupy the same position — between the topic and the table — so this is a choice per table, not a layering.



```
                    │                                 │       (via Polaris)
                    └─ Iceberg Kafka Connect sink ─┘
```

| | Flink | Spark Structured Streaming | Iceberg Kafka Connect sink |
|---|---|---|---|
| Achievable freshness | sub-second | minutes, set by the trigger | minutes, set by the commit interval (300,000 ms default) |
| Keyed state | full: joins, windows, event-time aggregation | available, in the micro-batch model | none; row-level transforms only |
| Upsert mechanism | upsert writes keyed on declared equality fields | full SQL `MERGE INTO`, copy-on-write or merge-on-read | simple upsert |
| Commit trigger | checkpoint, two-phase commit | one commit per micro-batch | fixed interval, coordinated by an elected worker over a control topic |
| Runtime required | its own cluster: JobManager, TaskManagers, checkpoint store | the Spark cluster the platform already runs | the Connect worker cluster already running Debezium |
| Available from | the streaming stage | **the foundation stage** | the streaming stage |

**Decide in this order.**

1. **Required freshness.** Sub-second admits only Flink. Minutes admits all three. Hours means this is not a streaming question at all — use scheduled batch (§6.4.3).
2. **Required state.** Joins, windows, aggregation, or deduplication across events rules out the Connect sink at any price; it cannot hold keyed state.
3. **Runtime already operated.** Where the first two leave a genuine choice, prefer the runtime the platform already runs. Adding a writer is cheaper than adding a cluster.

Applying that to Stratus: Spark arrives in the foundation stage while Flink and Kafka Connect both arrive in the streaming stage, so **minutes-latency ingestion is achievable on the current increment with no new runtime**. Spark micro-batch is therefore the default answer to most stated streaming requirements, and the other two are adopted for a specific, named reason.

**What the writer must do, whichever runtime provides it.** These are requirements on the ingestion tier, not implementation preferences, and they are the reason a supported runtime is used rather than a service written in-house:

- buffer to local disk rather than memory, so window size is not bounded by heap and a restart does not lose the in-flight window
- rotate on size or elapsed time, whichever comes first, writing one object per window per partition via multipart upload
- commit to the catalog, then commit source offsets — never the reverse (§6.4.4)
- on storage unavailability, stop committing and let consumer lag build; pause the consumer when the local buffer fills rather than failing, retrying inside the poll loop, or discarding data

That last point is what a writer tier exists for: it gives backpressure somewhere to go. A consumer calling the object store from inside its own poll loop has no buffer, so a storage stall forces it to block past the poll interval and be evicted from the group, crash-loop, or drop data. Eviction triggers a rebalance across every other consumer, whose replayed windows then hit the appliance simultaneously — the storage problem becomes a consumer-group problem that amplifies it. Two settings bound the residual coupling: event retention must cover the longest tolerable storage outage **plus** the throttled catch-up that follows it, which is usually the longer of the two; and the backbone must not use tiered storage on the same appliance, or an outage takes out both systems at once and the decoupling fails exactly when it is needed.

Window size belongs to the writer tier, never to the producing application. The producer cannot see target file size, consumer read patterns, or the appliance's available request capacity, and all three change independently of it. Worse, producer count is elastic: with N producer instances each buffering independently, one window yields N objects at roughly 1/N of the intended size, and scaling out under load multiplies object count precisely when the appliance is busiest. Write parallelism on the consumer side is bounded by partition count, which is chosen deliberately.

**Flink** is selected where freshness is genuinely sub-second or the job needs continuous keyed state. Its cost is an additional cluster with its own checkpoint store, savepoint lifecycle, and failure drills.

**Spark Structured Streaming** reuses the batch engine, its `MERGE INTO` support gives the strongest upsert story of the three, and one team skill set covers ingestion and the silver/gold transforms. Its floor is the trigger interval: it will not deliver sub-second, and a long-running streaming job has a different operational profile from a scheduled batch job even on the same cluster.

**The Iceberg Kafka Connect sink** removes the stream processor from the path entirely for tables that need no state, and its coordinator design addresses the commit-contention risk of §15.2 structurally: one elected worker commits, so *n* tasks across *m* intervals produce one snapshot per interval rather than *n* × *m*. Four costs must be accepted explicitly before adopting it:

- **Duplicates remain possible.** Exactly-once delivery is documented on KIP-447, but zombie fencing is not yet implemented. A task stalled beyond the consumer session timeout can have its partitions reassigned while still alive, and the zombie may then complete its own commit. Session-timeout tuning is a correctness control here, not merely rebalance hygiene.
- **The default failure posture is brittle for on-premises storage.** `iceberg.control.commit.max-consecutive-failures` defaults to **1**, so the coordinator terminates after a single failed commit, and connector errors are non-retryable, which fails the task. A storage blip that Flink or Spark would ride out by letting lag build will stop this connector. Raise it deliberately.
- **A silent misconfiguration exists.** If the Connect consumer group id and the control topic group id disagree, no coordinator is elected: records are consumed and nothing is ever committed, with no clear error. Any adoption must verify that a written record becomes a committed snapshot, not merely that the connector reports healthy.
- **Two offset stores.** Source offsets live in a sink-managed consumer group committed with the data-file events in one Kafka transaction; control topic offsets ride in the Iceberg snapshot and are read back so only later events commit. Resetting offsets means resetting both groups.

**What none of them change.** Polaris remains the catalog and governance is unaffected. The one-writer-per-table rule in §6 applies across all three: a table has one writer in steady state, whichever runtime it is, and a second writer added to an existing cluster is still a second writer. Orphan-file cleanup (§10.3) is required in every case, because all three can leave written-but-uncommitted files after a crash. Commit cadence drives the small-file tax (§6.4.5) identically. The sequence-number merge rule (§6.4.4) applies to any upsert path.

Connect sink configuration defaults were verified against the Iceberg Kafka Connect documentation on 2026-08-06; its coordinator, offset, and failure-recovery behaviour is drawn from the connector's design document and should be read in full before adoption.

References:

- Iceberg Kafka Connect sink: https://iceberg.apache.org/docs/latest/kafka-connect/
- Iceberg Spark structured streaming: https://iceberg.apache.org/docs/latest/spark-structured-streaming/

---

# 7. Control Planes

A clean design separates three planes.

## 7.1 Data Plane

Contains:

- Ceph RGW object storage
- Iceberg tables
- Apache Polaris REST catalog
- Spark
- Flink
- Kafka, Kafka Connect, and Debezium when required by the streaming use case
- Trino
- acceleration-layer serving access (Firebolt Core or ClickHouse) when deployed

## 7.2 Metadata and Governance Plane

Contains:

- Atlas
- Ranger
- glossary
- lineage
- classification
- ownership and stewardship

## 7.3 Orchestration and Operations Plane

Contains:

- Airflow
- platform automation
- retries and alerts
- table maintenance scheduling
- backfills
- promotions and approvals

This separation matters because failed platforms often blur these layers into one mess.

---

# 8. Catalog Strategy

A critical design decision is the Iceberg catalog strategy.

## Options evaluated

Common Iceberg catalog approaches include:

- filesystem catalog — file-path-based metadata, no multi-engine coordination, not suitable for a shared platform
- Hive catalog — legacy coupling to Hive Metastore, limits engine flexibility
- REST catalog — open API standard, engine-agnostic, fits multi-engine design
- JDBC catalog — simple but operationally fragile at scale; lacks REST API compatibility
- Nessie — REST-compatible with Git-like branching semantics; strong fit for data-environment workflows
- **Apache Polaris** — Apache-incubated open source REST catalog server; implements the Iceberg REST Catalog spec natively

Iceberg's Flink docs explicitly call out catalog configuration options such as `hive`, `rest`, `jdbc`, and other implementation-specific catalog types:

- Iceberg Flink catalog configuration: https://iceberg.apache.org/docs/latest/flink/

## Decision

**Apache Polaris** is the chosen REST catalog implementation for this architecture.

Reference:

- Apache Polaris: https://polaris.apache.org/
- Polaris GitHub: https://github.com/apache/polaris

## Why Apache Polaris

- implements the Iceberg REST Catalog open API specification natively — no vendor lock-in at the catalog layer
- Apache-incubated open source project with strong Iceberg ecosystem alignment
- designed as a multi-engine catalog; Spark, Flink, and Trino all connect via the standard REST Catalog interface
- enables zero-copy data sharing: every engine reads and writes the single physical copy of each Iceberg table in Ceph RGW through the catalog, with credential vending, so datasets are shared across engines and teams without exports, replicas, or per-engine copies. Iceberg's safe multi-engine table access is what makes this possible; Polaris provides the shared control point
- supports fine-grained access control at the catalog and namespace level
- on-prem deployable without commercial licensing requirements
- strategically cleaner than embedding catalog behavior in engine-local configurations
- keeps metadata control in a dedicated, independently operable service boundary

## Why not the alternatives

- **Filesystem catalog**: no shared multi-engine coordination, not suitable for a governed platform
- **Hive Metastore**: legacy dependency, limits engine flexibility, adds operational complexity
- **JDBC catalog**: operationally fragile at scale, lacks REST API compatibility, no access control model
- **Nessie**: strong Git-like branching semantics are valuable but add operational complexity that is not required in Phase 1; remains the preferred alternative if data-environment branching becomes a platform

requirement

### Alternative path

If the platform later needs Git-like branching, tagging, and data-environment workflows at the catalog layer — for example, isolated branch environments for engineering, staging, and production data — **Nessie** is the next candidate to evaluate. That should be treated as a deliberate platform decision, not an incremental implementation detail.

### Design rule

Spark, Flink, Trino, and all maintenance workflows must be configured to use the central Apache Polaris REST catalog. Engine-local catalog configurations that bypass Polaris are not permitted in governed environments.

### Production catalog requirements

The catalog is not just a lookup service. It is the control point for table namespace, current metadata location, transactional commits, and engine coordination. Production readiness therefore requires evidence for:

- catalog availability and latency under concurrent Spark, Trino, and maintenance access
- an approved external Polaris metadata store with backup, restore, and ownership defined
- auditability for namespace, table, credential, and metadata-location changes
- identity integration, scoped service principals, and a documented credential model
- restore drills that recover catalog state, Iceberg metadata files, manifests, and object data consistently
- compatibility checks for every engine that reads or writes governed tables

---

## 9. Governance Model

### 9.1 Minimum metadata standard for every dataset

Every bronze, silver, and gold dataset should have:

- business name
- technical name
- domain
- owner
- steward
- source system
- schema version
- sensitivity / classification
- SLA / refresh expectation
- quality status
- retention rule
- downstream usage tags

### 9.2 Lineage expectations

Lineage should cover:

- source files / streams to bronze
- bronze to silver transformations

- silver to gold materialisations
- consuming marts and serving tables

### 9.3 Lineage delivery contract

Lineage quality will be poor unless metadata emission is standardized. Spark and Flink jobs should emit dataset, schema, run, and transformation metadata in a consistent format, and publication to Atlas should be part of deployment and job-completion contracts rather than a best-effort extra.

### 9.4 Domain ownership

The platform team should own the shared platform and conventions. Domain teams should own their data products.

That split is important. Otherwise the central team becomes a bottleneck and the platform becomes theatre.

---

## 10. Orchestration Model

### 10.1 Airflow should orchestrate

- Spark batch jobs
- data quality tasks
- metadata publication tasks
- promotion workflows
- Iceberg maintenance jobs
- periodic compaction windows
- snapshot retention enforcement

Airflow DAGs should be structured around platform outcomes, not individual shell commands. A typical DAG should read as:

```
detect input → run Spark job → run quality checks → evaluate promotion → publish metadata → notify
```

Each task should emit structured log fields:

| Field | Purpose |
|---|---|
| `run_id` | stable pipeline run identifier shared across Airflow, Spark, quality, and lineage |
| `dataset` | fully qualified Iceberg table name |
| `zone` | bronze / silver / gold / platform |
| `spark_application_id` | Spark application id for debugging |
| `quality_status` | passed / failed / warning / overridden |
| `promotion_decision` | promote / block / overridden |

#### 10.1.1 DAG ownership and source control

All DAGs should be version-controlled with the application code and reviewed like production code. DAG owners must be named explicitly. A DAG without a clear owner, retry policy, alert route, and expected runtime should not be promoted to production.

### 10.1.2 Retry and backfill semantics

Retries are for transient infrastructure or source availability failures. They must not hide deterministic data failures such as schema incompatibility, failed quality checks, or authorization denial.

Backfills must be idempotent. Re-running a DAG for a historical date should either produce the same target table state or create a clearly versioned replacement snapshot. Backfills should not silently append duplicate business records.

## 10.2 Flink should execute continuously

- streaming jobs remain long-running where appropriate
- operational lifecycle for Flink jobs should be separate from normal Airflow-style task semantics

## 10.3 Table maintenance

Iceberg maintenance is not optional.

The platform must schedule:

- snapshot expiration — target cadence: daily for active tables, weekly for archival
- compaction / file rewrite — target: keep file counts within 2x the optimal range for each table's typical query pattern
- orphan file cleanup — run at least weekly; alert if orphan volume exceeds a configurable threshold
- metadata health checks — verify manifest list depth, metadata.json file count, delete-file count, and manifest growth; compact metadata when thresholds are breached

Maintenance must be metadata-driven. Jobs should inspect Iceberg metadata tables such as `files`, `snapshots`, `manifests`, and `history` before deciding whether to compact, expire snapshots, or raise an alert. Fixed schedules are acceptable as triggers, but the action taken should depend on table-specific policy and current table state.

Each governed table must have a maintenance policy before production onboarding. The policy must define at least:

| Policy field | Purpose |
|---|---|
| table owner and maintenance owner | identifies who approves thresholds and who responds to alerts |
| target file size and minimum average file size | drives rewrite/compaction decisions from the `files` metadata table |
| maximum file count and small-file count | prevents unbounded small-file growth and planning overhead |
| maximum snapshot-chain length and snapshot retention | drives snapshot expiry from the `snapshots` and `history` metadata tables |
| maximum manifest count or manifest growth rate | drives manifest rewrite or alerting from the `manifests` metadata table |

| Policy field | Purpose |
|---|---|
| maximum delete-file count | triggers delete-file cleanup or rewrite for tables with row-level deletes |
| orphan-file threshold and cleanup cadence | controls cleanup and alerting for unreferenced objects |
| alert severity and allowed maintenance window | prevents disruptive maintenance during critical serving windows |

**Ownership**: The platform / infrastructure team owns the maintenance jobs and their scheduling. Domain teams do not run ad-hoc maintenance. Compaction ownership must be explicitly assigned per table (normally the platform team); no table should be left without a designated maintenance owner.

**Targets are starting points.** Adjust cadences based on observed table activity and query performance. The key discipline is that every table has a maintenance schedule and an owner — not the specific numbers.

**Event retention is a maintenance parameter too, and it is sized against replay rather than against the outage.** Retention must cover the outage plus the throttled catch-up that follows, and the catch-up is normally the longer term: with replay throttled to $k\times$ the ingest rate, retention must be at least $T \times k/(k-1)$ for an outage of $T$. A six-hour maintenance window with replay throttled to 1.5× ingest therefore requires eighteen hours of retention, not six. The two settings pull against each other — throttling replay harder to protect the storage tier increases the retention required to survive the same outage — so the throttle is set first from the measured operating budget (§6.4.5) and retention is sized from it. Undersized retention converts consumer lag into permanent data loss.

If this is neglected, performance and reliability will degrade over time.

## 10.4 Data quality and promotion

Quality check execution, result storage, promotion gates, override handling, and ownership are fully specified in **§5 Data Quality Subsystem**.

## 10.5 Control-plane state

The following are control-plane state and must be protected by backup, restore, and change-control procedures:

| Component | State to protect |
|---|---|
| Airflow | metadata database, DAG code, connection definitions, variables, task logs |
| Polaris | catalog metadata store and principal/role configuration |
| Atlas | graph store, search index, type definitions, glossary, classifications |
| Ranger | policies, tag policies, audit configuration, usersync configuration |
| FreeIPA | Kerberos principals, LDAP users/groups, DNS, PKI material |
| Keycloak | realms, clients, identity mappings, token configuration |

If control-plane state is not backed up, the platform is not recoverable even if the data files in object storage survive.

---

# 11. Reference End-to-End Flow

Write modes referenced below are defined in §6.4.

## 11.1 Batch flow

1. source files arrive in the object-storage landing zone, or a bounded extract is captured per §6.1
2. Airflow triggers ingestion and validation pipeline
3. Spark normalises and **appends** to bronze Iceberg tables — bronze is never mutated in place
4. Spark transforms bronze to silver, **upserting copy-on-write** where change rates are low
5. Spark or SQL jobs materialise gold tables by **copy-on-write upsert or full rebuild**
6. Atlas metadata and lineage are updated
7. the acceleration engine (Firebolt Core or ClickHouse) optionally serves curated gold datasets

Commit cadence is the job schedule, so the small-file pressure of §6.4.5 is low and compaction runs on its ordinary maintenance schedule.

## 11.2 Streaming flow

This flow applies when a streaming backbone such as Kafka is deployed (typically Phase 2+).

1. source database changes are captured by a Debezium connector running in Kafka Connect and published to a Kafka topic, carrying before/after images and a sequence number
2. business events from application producers land directly in Kafka topics
3. Flink consumes Kafka topics, enriches and transforms streams; topics are keyed on the entity primary key so all changes to one row stay ordered in one partition
4. Flink **appends** change events to bronze Iceberg tables continuously via the Polaris catalog, committing on the checkpoint interval
5. current-state silver tables are produced either by **merge-on-read upsert** from Flink, or by the **buffer-then-merge** shape — periodic MERGE from the bronze change stream — which is the preferred default because it keeps ingestion append-only
6. downstream Spark or Flink jobs materialise higher-order views
7. Atlas metadata and lineage are synchronised
8. the acceleration engine or BI consumers query curated outputs via Trino or directly

Two obligations attach to this flow specifically: merges compare the source sequence number so late events cannot overwrite newer state (§6.4.4), and compaction must keep pace with delete-file growth or read latency degrades while writes continue to look healthy (§6.4.2).

Steps 3 and 4 assume Flink between the topic and the table. Spark Structured Streaming and the Iceberg Kafka Connect sink can occupy that same position for tables whose freshness requirement is minutes rather than sub-second; §6.4.7 compares the three writers.

---

# 12. Physical Deployment View

A realistic on-prem deployment would separate infrastructure by concern.

## 12.1 Storage tier

- Ceph RGW object store backed by a Ceph cluster
- erasure coding / replication depending on product choice

- separate buckets or namespaces by domain and lifecycle zone

### 12.2 Compute tier

- Spark cluster for batch compute
- Flink cluster for stateful stream processing
- Firebolt Core or ClickHouse nodes if the acceleration layer is deployed

### 12.3 Control tier

- Airflow API server, DAG processor, scheduler, triggerer, local executor, and metadata DB (PostgreSQL)
- Apache Atlas developer profile with disposable embedded dependencies; production profile with external HBase, SolrCloud/ZooKeeper, and an external supported notification service
- FreeIPA identity services (Kerberos KDC, LDAP, DNS, PKI)
- Keycloak OIDC broker
- Apache Ranger admin server and usersync service
- monitoring and logging stack

#### 12.3.1 Control-tier availability posture

Phase 1 may run single instances of Polaris, Atlas, Ranger, Airflow, and Keycloak for lab and early platform validation. Production must define recovery objectives for each control-plane service before onboarding critical datasets.

Minimum production posture should include:

- PostgreSQL backup and restore testing for Airflow and any catalog or governance metadata stores that use PostgreSQL
- exported Keycloak realm configuration and FreeIPA backup procedures
- documented rebuild path for every OCI image and runtime unit
- persistent volumes outside container writable layers
- restore drills before sensitive or business-critical datasets are onboarded

### 12.4 Platform services

- Apache Polaris REST catalog service
- FreeIPA identity services (Kerberos KDC, LDAP, DNS, PKI)
- Keycloak OIDC broker
- secrets management
- `platform.quality_check_results` Iceberg table (owned by platform team)
- observability stack (see §14.1)
- certificate and TLS management via FreeIPA Dogtag PKI

---

## 13. Security and Access Model

### 13.1 Identity foundation

The platform is Linux-only. The identity stack is:

| Component | Role |
|---|---|
| **FreeIPA** | core identity provider — Kerberos KDC, LDAP directory, DNS, PKI |
| **Keycloak** | OIDC broker for REST-facing services — backed by FreeIPA |
| **MIT Kerberos client** | installed on all compute nodes for ticket-based service authentication |
| **SSSD** | Linux host integration with FreeIPA for OS-level authentication |

### Why FreeIPA

FreeIPA bundles MIT Kerberos, 389 Directory Server (LDAP), Dogtag PKI, and DNS into a single managed identity service designed for Linux-native on-prem deployments. It eliminates the need for Windows Active Directory entirely while providing the same Kerberos and LDAP interfaces that Ranger, Atlas, Spark, and Flink expect.

Reference:

- FreeIPA: https://www.freeipa.org/

### Why Keycloak

Keycloak is an open source OIDC/SAML identity broker. It sits in front of FreeIPA and provides token-based authentication for REST-facing platform services — Apache Polaris, Airflow web UI, and any data API layer. This avoids embedding Kerberos ticket handling into REST clients while keeping FreeIPA as the single source of truth for principals and groups.

Reference:

- Keycloak: https://www.keycloak.org/

---

## 13.2 Authentication model by component

| Component | Authentication mechanism |
|---|---|
| Spark jobs | approved service identity; Kerberos keytab where the selected runtime integration uses Kerberos |
| Flink jobs | future service identity; Kerberos keytab where the selected runtime integration uses Kerberos |
| Airflow workers | approved service identity for job submission |
| Apache Polaris | OIDC token via Keycloak where supported by the selected Polaris release |
| Trino | HTTPS and OIDC via Keycloak for client access; internal trust via the selected Trino secure-communication model |
| Atlas | LDAP via FreeIPA for user authentication |
| Ranger | LDAP via FreeIPA for user/group sync; HTTPS policy download to engine plugins |
| Ceph RGW | service accounts with access/secret key pairs; TLS enforced |
| Airflow web UI | OIDC via Keycloak |

### 13.2.1 Service and protocol interaction model

Security in Stratus is not one mechanism. Each protocol has a specific job:

| Protocol | Platform role |
|---|---|
| DNS | stable service identity, certificate SAN alignment, and Kerberos host canonicalization |
| Kerberos | Linux and service principal authentication where ticket-based service identity is implemented |
| LDAP / LDAPS | FreeIPA user and group lookup for Keycloak, Ranger usersync, Atlas, and SSSD |
| OIDC / OAuth2 | browser, CLI, and REST-facing service authentication through Keycloak |
| HTTPS / TLS | transport encryption and server identity for every platform endpoint |
| S3 API over HTTPS | object access to Ceph RGW for Spark, Trino, Polaris, Airflow, and verification tools |
| Iceberg REST catalog over HTTPS | table and namespace resolution through Polaris |
| Ranger policy REST | policy download from Ranger Admin to the Trino Ranger plugin |
| JDBC over HTTPS | analyst, BI, and verification access through Trino |

The core identity chain is:

```
FreeIPA users/groups
      ├── LDAPS ──► Keycloak ──OIDC tokens──► Trino / Airflow / Polaris where
supported
      ├── LDAPS ──► Ranger usersync ──policies──► Trino Ranger plugin
      ├── LDAPS ──► Atlas authentication
      └── SSSD/Kerberos ──► Linux host and service identities
```

The core data access chain is:

```
User / BI tool ──OIDC/JDBC──► Trino
Trino ──Ranger policy check──► Ranger
Trino ──Iceberg REST──► Polaris
Trino ──S3 API──► Ceph RGW
```

Ranger governs what an authenticated user can query through Trino. Polaris governs catalog, namespace, and table metadata resolution. Ceph RGW governs service-account object access. These layers are complementary and must not be collapsed into a single bucket policy or a single OIDC login check.

### Service account model

Every pipeline component runs as a named Linux service account registered in FreeIPA. No shared credentials. No human credentials used for job execution. Keytabs are managed centrally and rotated on a defined schedule.

### Secrets handling

Service credentials must not be committed to source control or embedded directly in DAG code, Spark code, Dockerfiles, or documentation examples used as live configuration. Early increments may use environment files on secured hosts, but the target model is:

- credentials stored in a dedicated secrets manager or equivalent secured platform service
- Airflow reads credentials through a secrets backend or protected connections
- Spark and Flink jobs receive short-lived or centrally rotated service credentials
- Ceph RGW access keys, Polaris client secrets, Keycloak client secrets, and Kerberos keytabs have named owners and rotation schedules
- every secret has a documented consumer list and emergency rotation procedure

Plaintext environment files are acceptable only as a lab bootstrap mechanism and must be treated as temporary.

---

## 13.3 Authorisation and policy enforcement

The platform uses a two-layer authorisation model:

**Layer 1 — Ranger policy enforcement** Ranger enforces data access policy at the engine layer for Spark, Flink, Trino, and Atlas. Ranger's `usersync` service polls FreeIPA LDAP on a schedule to import users and groups. Policies are defined against FreeIPA groups, not individual users.

**Layer 2 — Polaris catalog access control** Apache Polaris enforces catalog-level access at the namespace and table level. Polaris principals are mapped to Keycloak identities. Polaris roles control which principals can read, write, or manage catalog namespaces and tables.

These two layers are complementary:

- Polaris controls what the catalog exposes to each engine identity
- Ranger controls what each engine identity can do with the data once the catalog resolves it

---

## 13.4 Group and role model

FreeIPA groups are the unit of policy assignment. Ranger and Polaris policies are written against groups, not individuals.

Recommended group structure:

| Group | Access scope |
|---|---|
| `platform-admins` | full platform administration |
| `platform-engineers` | pipeline development and deployment |
| `data-stewards-<domain>` | domain metadata management and quality override authority |
| `analysts-<domain>` | read access to silver and gold datasets for their domain |
| `consumers-gold` | read access to curated gold datasets across domains |
| `svc-spark` | Spark service account group |
| `svc-flink` | Flink service account group |

| Group | Access scope |
|---|---|
| `svc-airflow` | Airflow service account group |

Group membership is managed in FreeIPA. Policy assignment is managed in Ranger and Polaris. The two concerns are kept separate.

---

### 13.5 Classification-driven access control

Atlas classifications are the bridge between governance metadata and Ranger enforcement:

1. Atlas classifies a dataset (e.g. `PII`, `CONFIDENTIAL`, `RESTRICTED`)
2. Ranger tag-based policies apply access rules to any dataset carrying that classification
3. Access is enforced automatically when a new dataset is classified — no manual policy update per table required

This means sensitive datasets are protected by classification, not by remembering to write a new Ranger rule for each table. The enforcement model scales with the data catalogue.

---

### 13.6 Encryption

- **In transit**: TLS enforced across all inter-service communication; certificates issued by FreeIPA's Dogtag PKI
- **At rest**: use the approved Ceph/RGW encryption-at-rest model for sensitive zones
- **Kerberos tickets**: short-lived; keytab rotation managed via FreeIPA

---

### 13.7 Minimum access control requirements

The platform must enforce:

- role-based access by domain and environment, backed by FreeIPA groups
- separation between raw (bronze), curated (silver/gold), and sensitive zones enforced by Ranger
- service-account-based job execution — no human credentials in pipelines
- encryption in transit and at rest
- audited data access via Ranger audit logs
- classification-driven policy attachment via Atlas + Ranger tag-based policies

Sensitive datasets must not rely on bucket naming and tribal process alone. Classification, policy, and enforcement must be aligned from day one.

---

### 13.8 Source references

- FreeIPA: https://www.freeipa.org/
- Keycloak: https://www.keycloak.org/
- Apache Ranger LDAP usersync: https://ranger.apache.org/
- MIT Kerberos: https://web.mit.edu/kerberos/

---

## 14. Operational Model

## 14.1 Observability

**Tooling**

The observability stack uses open source components consistent with the Linux-only, no-proprietary-dependency constraint:

| Concern | Tooling |
|---|---|
| Metrics collection and storage | **Prometheus** |
| Dashboards and alerting | **Grafana** |
| Log aggregation | **Grafana Loki** (or OpenSearch if full-text search of logs is required) |
| Distributed tracing (optional) | **Grafana Tempo** |

All four are open source (Apache 2.0 / AGPLv3), on-prem deployable, and integrate natively with each other. Prometheus exporters or metrics integrations exist for Spark, Flink, Kafka, Ceph, Airflow, Trino, and the JVM-based services (Atlas, Polaris, Ranger).

References:

- Prometheus: https://prometheus.io/
- Grafana: https://grafana.com/oss/grafana/
- Grafana Loki: https://grafana.com/oss/loki/

**What to track**

- pipeline success/failure rates and durations (Airflow DAG metrics via Prometheus StatsD exporter)
- end-to-end data freshness per dataset (derived from Atlas `quality_last_checked` and Iceberg snapshot timestamps)
- Flink job lag and checkpoint health (Flink metrics exposed via Prometheus reporter)
- Spark job duration and failure causes (Spark metrics sink to Prometheus)
- Iceberg metadata growth — manifest list depth and metadata file count per table
- file-count explosion and compaction debt (query `platform.quality_check_results` and Iceberg table metrics)
- Kafka consumer lag for Flink source topics (Kafka exporter to Prometheus)
- serving-layer query latency (Trino JMX metrics via Prometheus)
- metadata publication freshness in Atlas
- Ranger audit log volume and policy evaluation latency
- DAG import errors and scheduler heartbeat freshness
- promotion gate block rate and override frequency
- Ceph RGW bucket growth by zone and dataset
- Polaris catalog request latency and authentication failures
- control-plane backup age and last restore-test status

### 14.1.1 Logging contract

Task logs and service logs should be structured enough to correlate a platform run across Airflow, Spark, Polaris, Ceph RGW, and Atlas.

At minimum, logs emitted by pipeline jobs should include:

- `run_id`
- `dag_id` or job name
- source dataset and target dataset
- Iceberg snapshot id after write
- Spark application id where applicable
- quality status and promotion decision
- exception class and failure category on error

Logs should avoid printing secrets, tokens, access keys, or full connection strings.

## 14.2 Reliability disciplines

- explicit retry policies
- backfill procedures
- replay procedures for bronze data
- change management for schema evolution
- controlled promotion between environments
- backup and restore procedures for control-plane metadata
- idempotent job design for scheduled and manually retried workflows
- documented incident response for failed quality gates, stale datasets, and table maintenance failures

### 14.2.1 Recovery expectations

The platform should define recovery expectations separately for data and control-plane state:

| Area | Recovery expectation |
|---|---|
| Ceph data | recover from node or drive loss according to replication or erasure-coding policy |
| Iceberg tables | recover through snapshots, rollback, and retained metadata |
| Airflow | restore metadata DB and DAG code sufficiently to resume scheduling with run history |
| Polaris | restore catalog metadata so table identifiers continue resolving to the same locations |
| Atlas and Ranger | restore lineage, classifications, policies, and audit configuration |
| Identity | restore FreeIPA and Keycloak without changing service identities unexpectedly |

Recovery drills should verify that restored services can run an end-to-end bronze-to-gold workflow, not merely start containers.

## 14.3 Data quality

Quality execution, result storage, promotion gates, and override handling are fully specified in **§5 Data Quality Subsystem**. Operationally, monitor the `platform.quality_check_results` Iceberg table for failure trends, override frequency, and check coverage per domain.

## 14.4 Cross-increment QA traceability

Each implementation increment must leave behind verified platform capability that the next increment consumes. Verification should therefore test both the new component and the inherited contracts from previous increments.

| Increment | Produces | Consumed by | Cross-check required |
|---|---|---|---|
| 1 — Storage | TLS S3 endpoint, five platform buckets, service accounts, bucket policies | Polaris, Spark, Airflow, Trino | bucket existence, path-style S3 access, credential isolation, HTTPS-only access, Ceph health |
| 2 — Iceberg and Polaris | `stratus` catalog, bronze/silver/gold/platform namespaces, `platform.quality_check_results` | Spark, Airflow, Trino, quality subsystem | namespace resolution, table creation, table reads/writes, quality table schema |
| 3 — Spark | ingestion, transform, materialisation, quality, promotion, and maintenance jobs | Airflow orchestration and Trino result validation | Spark can read/write via Polaris and Ceph RGW; quality records are written with run IDs and snapshot IDs |
| 4 — Airflow | scheduled DAGs, retries, alerts, promotion gates, maintenance orchestration | Trino query validation and operational monitoring | DAGs submit Spark jobs, failed blocking checks halt downstream tasks, maintenance runs on schedule |
| 5 — Trino | shared SQL query plane over governed Iceberg tables | analysts, BI, governance validation | SQL results match Spark outputs; Trino resolves tables through Polaris and does not bypass the catalog |
| 6 — Atlas and Ranger | metadata entities, lineage, classifications, policies, and audit | identity hardening and secure operations | Atlas metadata is searchable; Ranger allow/deny behavior works through Trino |
| 7 — FreeIPA and Keycloak | identity, OIDC, Kerberos, LDAP groups, trusted certificates | secure platform operations | earlier increment behavior still passes after replacing lab users, self-signed certificates, and bootstrap credentials |

This traceability prevents each increment from becoming a local installation exercise. For example, Increment 5 should not merely prove that Trino starts. It should prove that Trino can query the exact Iceberg tables produced by Spark, orchestrated by Airflow, registered in Polaris, and stored in Ceph RGW.

## 14.5 End-to-end functional assertions

The platform should maintain a small permanent verification dataset that can be recreated safely in every environment. It should exercise the full batch path:

```
landing file → Spark ingestion → bronze table → quality checks → Airflow gate
             → silver table → quality checks → Airflow gate
             → gold table → Trino query
```

The following assertions should be stable across increments:

| Assertion | Why it matters |
|---|---|
| landing file is readable only by intended service accounts | validates storage service-account policies from Increment 1 |
| bronze table preserves raw row count, including intentional duplicate | validates ingestion fidelity from Increment 3 |
| quality check records both warning and blocking outcomes | validates the quality result contract from Increment 2 and 3 |
| Airflow blocks promotion on a failed blocking check | validates orchestration behavior from Increment 4 |
| silver table contains deduplicated rows only after the gate permits promotion | validates promotion sequencing and transform correctness |
| gold aggregate matches expected grouped output | validates materialisation correctness |
| Trino returns the same counts and aggregates as Spark | validates Increment 5 as an independent query plane |
| query against a nonexistent column fails clearly | validates schema enforcement and user-facing error behavior |

## 14.6 QA ownership

QA for the platform is not a separate after-the-fact activity. Each platform layer owns its verification contract:

| Area | Owner | Verification mechanism |
|---|---|---|
| Storage | platform infrastructure | S3 SDK tests and Ceph/RGW operational checks |
| Catalog and tables | platform data architecture | Iceberg Java API tests and Polaris API checks |
| Batch compute | data engineering platform | Spark verification jobs and table assertions |
| Orchestration | platform operations | Airflow REST API tests and DAG-state assertions |
| Query | analytics platform | Trino JDBC tests and SQL result checks |
| Governance | data governance | Atlas entity checks and Ranger policy tests |
| Identity | platform security | Kerberos/OIDC authentication and authorization tests |

Every completion gate should have an automated verification suite where practical and an operational checklist for conditions that cannot be fully automated in the lab.

Phase 1 operational acceptance is captured in stratus_phase1_operational_readiness.md. That document is the closeout gate for Increments 1 through 7: it verifies integrated function, recovery, observability, runbooks, security posture, governance controls, quality gates, and operational ownership before production dataset onboarding or Phase 2 work begins.

## 14.7 Upstream reference audit and version discipline

Reference baseline: 2026-08-25.

The platform depends on fast-moving open source projects. Each implementation increment must start by checking current upstream documentation and release notes for the selected versions, then recording the approved version matrix in the increment runbook. A design document may intentionally pin an older version for compatibility, but that pin must be explicit and verified.

Minimum version matrix to maintain:

| Component | Version discipline |
|---|---|
| Java | use Java 21 LTS for Stratus-owned builds, verifier images, and component runtimes where supported; pin the vendor and latest approved Java 21 patch; component-mandated exceptions require an explicit owner, version pin, evidence, and removal trigger |
| Ceph RGW | choose a supported Ceph release and pin all images/packages; do not use `latest` |
| Apache Polaris | pin the catalog release and validate REST catalog behavior against Iceberg clients |
| Apache Iceberg | align Java API, Spark runtime, Flink runtime, and Trino connector expectations |
| Apache Spark | align Spark major version, Scala version, and Iceberg Spark runtime artifact |
| Apache Airflow | standardize on Airflow 3.x and keep API server, DAG processor, scheduler, triggerer, provider packages, and auth-manager behavior aligned |
| Trino | keep coordinator, workers, JDBC driver, Iceberg connector behavior, and Ranger plugin config on the same release line |
| Apache Atlas / Ranger | build pinned platform images from approved Apache releases and record Java/database/plugin compatibility |
| Kafka / Kafka Connect / Debezium | align Kafka broker/KRaft mode, Connect worker version, and Debezium connector series |
| Apache Flink | align Flink major version, Java support, connectors, checkpointing, and Iceberg runtime |
| FreeIPA / Keycloak | use current identity-provider documentation for LDAP/Kerberos/OIDC integration and avoid stale user-guide assumptions |

Current Phase 1 target baseline as of 2026-08-25:

| Component | Target |
|---|---|
| Stratus Java build and verifier baseline | Java 21 LTS, latest approved patch |
| Java build tool | Apache Maven 3.9.16; Maven 4 remains pre-GA and is not the production build baseline |
| OCI runtime baseline | Podman 5.8.2 preferred; Docker Engine 29.5.3 permitted where selected and component-supported; exact package and patch pinned per environment |
| Component runtime exceptions | Spark 4.1 and Airflow's Spark client use the Java 21 baseline; Trino 483 requires Java 25.0.1 or newer Java 25 and remains an explicit component exception; Atlas/Ranger use their selected release's supported runtime; all exceptions are pinned and recorded |
| Apache Polaris | 1.5.0 |
| Apache Iceberg | 1.11.0 |
| Apache Spark | 4.1.2 with Scala 2.13 |
| Apache Airflow | 3.3.1 |
| Airflow Python runtime | Python 3.14; the official Airflow 3.3.1 image currently embeds 3.14.3 while upstream 3.14.6 remains a monitored base-image update |
| Airflow metadata database | PostgreSQL 17.10, retained approved matrix entry within Airflow 3.3.1's tested PostgreSQL 14-18 range |
| Airflow Spark provider | 6.3.1 |
| Airflow Amazon provider | 9.34.0 |
| boto3 | 1.43.56 |
| Trino | 482 |
| Keycloak | 26.6.4 |
| Keycloak metadata database | PostgreSQL 18.4, latest patch in Keycloak's newest supported PostgreSQL major |
| Ceph RGW | Ceph Tentacle 20.2.2, verified as the current Tentacle patch on 2026-07-14; pin by package version or image tag plus digest |
| Apache Atlas | 2.5.0, built as an internal image and pinned by tag plus digest after dependency compatibility review |
| Apache Ranger | 2.8.0, built as an internal image and pinned by tag plus digest after plugin/database compatibility review |
| FreeIPA | approved package stream from the selected Linux distribution, pinned by repository/channel and package version in the environment version matrix |

Spark 4.2.0 is treated as preview and is not the Phase 1 production target until it becomes a stable release and the Iceberg runtime, Airflow Spark provider, Trino connector, and verification suites are updated together.

Airflow 3.3.1's official Python 3.14 constraints still select Spark provider 6.3.1, Amazon provider 9.34.0 and boto3 1.43.56. Spark provider 6.3.2 and Amazon provider 9.35.0 were released on 2026-08-23. They remain upgrade candidates, not piecemeal replacements: promotion requires a regenerated dependency lock plus image, import, Spark-submission, landing-sensor and public-API regression evidence.

No implementation increment should be signed off with floating container tags, unverified compatibility assumptions, or examples copied from quickstarts without adapting them to the Stratus security and QA model.

### 14.8 Build, artifact, and runtime separation

Stratus builds software and container images in the approved build system. Runtime and verification environments consume immutable, versioned artifacts; they do not compile source code or resolve build dependencies at execution time.

The following contract applies to every increment:

- application JARs, verifier JARs, Python distributions, plugins, and custom container images are built, tested, scanned, and published by the build system
- every deployed artifact is identified by version plus checksum or image digest and is recorded in the evidence bundle
- runtime and verification containers contain only the runtime, deployed artifact, configuration, certificates, and narrowly scoped credentials required to execute their role
- source trees, Maven or Gradle project directories, compiler toolchains, and writable dependency caches are not mounted into runtime or verification containers
- verification is performed by deploying a pinned verifier container image that contains the prebuilt verifier artifact and compatible runtime, then executing it against the target environment
- verification results are written to a dedicated evidence volume or collected from standard output; the application artifact and runtime filesystem remain read-only where practical
- Dockerfiles and Containerfiles are build inputs executed by the build system, never ad hoc production-host build instructions
- `spark-submit`, Flink job submission, Airflow task execution, and similar commands must reference an already-built, checksummed job artifact
- the standard Stratus build JDK, verifier runtime, and supported component runtime is Java 21 LTS at the latest approved patch level
- a component may use a different Java release only when its selected upstream release requires it; the exception, owner, pinned runtime, evidence, and removal trigger must be recorded in the version matrix

An increment document may show a build-system command such as `mvn verify` only in a clearly labelled build/publish stage. Its deployment and acceptance steps must execute the published verifier image without invoking Maven, Gradle, a compiler, or a package build. Verifier images use the verifier artifact as their entrypoint, accept configuration through a protected environment file or approved secret injection, mount trust material read-only, and write only to a dedicated evidence mount.

---

## 15. Risks and Mitigations

### 15.1 Risk: file swamp instead of platform

**Cause:** raw files treated as the contract rather than Iceberg tables.

**Mitigation:** mandate Iceberg for all governed analytical datasets.

### 15.2 Risk: Spark and Flink commit contention or ownership confusion

**Cause:** multiple engines writing the same tables without clear rules. Iceberg commits are optimistically concurrent: writers that touch overlapping files detect the conflict at commit time, retry, and under sustained contention can livelock, so throughput collapses while both engines appear to be running normally.

**Mitigation:** define one-writer-per-table steady-state ownership where possible, assign compaction ownership explicitly, and select the write mode per table as defined in §6.4 rather than letting it follow from engine choice. Note that the Iceberg Kafka Connect sink (§6.4.7) addresses this risk structurally, by electing a single coordinator so that many tasks produce one snapshot per interval instead of one per task. Where a second writer is unavoidable, partition writers by key range so they never touch the same files, or serialise commits per table. Copy-on-write upserts contend most, because rewriting a file conflicts with any other writer touching it; append-only and merge-on-read writers produce disjoint files and contend least.

#### 15.2.1 Risk: merge-on-read tables degrade silently for readers

**Cause:** delete files accumulate faster than compaction removes them. Write latency stays flat, so the writing team sees no symptom while consumer query latency rises continuously.

**Mitigation:** treat compaction as a scheduled pipeline stage rather than cleanup, alert on the delete-file count threshold already required by the §10.3 maintenance policy, and monitor read latency as a table-level signal independent of write health.

### 15.3 Risk: Atlas becomes shelfware

**Cause:** no serious metadata ingestion, no ownership discipline, no lineage automation.

**Mitigation:** make metadata publication mandatory in delivery pipelines, standardize metadata emission from Spark and Flink jobs, and establish data stewardship roles.

### 15.4 Risk: Airflow becomes a platform dumping ground

**Cause:** every dependency and runtime concern pushed into DAGs.

**Mitigation:** keep Airflow focused on orchestration and control-plane workflows.

### 15.5 Risk: acceleration layer adopted too early

**Cause:** performance tooling added before storage, catalog, and governance foundations are stable.

**Mitigation:** keep the acceleration layer (Firebolt Core or ClickHouse) optional and phase it after core platform maturity.

### 15.6 Risk: too much platform, not enough product value

**Cause:** architecture overbuild without domain-aligned data products.

**Mitigation:** deliver by domain use case, with measurable product outcomes and curated gold datasets.

### 15.7 Risk: platform ambiguity at the seams

**Cause:** catalog undecided, query plane undecided, authorization undecided, lineage automation assumed.

**Mitigation:** treat catalog, query plane, authorization model, and lineage publication as named platform products with explicit owners and operating expectations.

---

# 16. Phased Roadmap

## Phase 1 – Foundation

Deliver:

- Ceph RGW object storage
- Iceberg table standard
- Apache Polaris REST catalog
- Spark batch processing
- Trino shared query layer
- Airflow orchestration
- Atlas and Ranger baseline governance and policy model
- bronze/silver/gold standards

Primary outcome:

- governed batch lakehouse foundation

Phase 1 acceptance:

- completed through the operational readiness gate in stratus_phase1_operational_readiness.md, which validates integrated function, recovery, observability, security, governance, quality, and runbook ownership

## Phase 1 scope note

Phase 1 is the fixed Increment 1 through Increment 7 foundation. Developer-profile completion may proceed while production dependencies are being provisioned, but no named component is silently deferred and the production-readiness gate remains blocked until every production profile passes.

## Phase 2 – Streaming and Operational Maturity

Deliver:

- Kafka event backbone
- Kafka Connect worker cluster
- Debezium CDC connectors for source systems
- Flink streaming ingestion
- CDC pipelines
- Atlas notification traffic consolidated onto the platform Kafka backbone, replacing the Phase 1 dedicated service or developer embedded notifier
- lineage automation improvements
- streaming-aware Iceberg maintenance and commit coordination

Primary outcome:

- near-real-time ingestion and processing

## Phase 3 – Query Acceleration and Data Products

Deliver:

- acceleration serving layer (Firebolt Core or ClickHouse) if justified
- curated business marts
- semantic serving views
- domain-owned data products

Primary outcome:

- low-latency consumption and broader business adoption

### Phase 4 – Advanced Governance and Self-Service

Deliver:

- stronger glossary alignment
- policy-driven classification workflows
- self-service dataset discovery and onboarding
- reusable domain patterns and templates

Primary outcome:

- scalable enterprise data fabric operating model

---

## 17. Recommended Final Position

The recommended architecture is:

**Data layer**

- **Ceph RGW** for durable object storage
- **Apache Iceberg** as the mandatory table abstraction and data contract
- **Apache Polaris** as the central REST catalog and multi-engine metadata control point

**Compute layer**

- **Apache Spark** for heavy batch compute and historical transforms
- **Apache Flink** for streaming and real-time computation
- **Trino** as the default open interactive SQL query plane
- **Firebolt Core** or **ClickHouse** as an optional serving/query acceleration layer over curated Iceberg datasets

**Streaming and CDC layer** (Phase 2+)

- **Apache Kafka** as the durable event backbone
- **Kafka Connect** as the connector framework for source system integration
- **Debezium** as the CDC connector for database change capture

**Governance layer**

- **Apache Atlas** for metadata, glossary, lineage, and classifications
- **Apache Ranger** for policy enforcement tied to Atlas classifications and access rules

**Orchestration layer**

- **Apache Airflow** for batch orchestration, promotion gates, and control-plane automation

**Identity and security layer**

- **FreeIPA** as the Linux-native identity provider (Kerberos, LDAP, PKI)
- **Keycloak** as the OIDC broker for REST-facing services

**Observability layer**

- **Prometheus + Grafana** for metrics and dashboards
- **Grafana Loki** for log aggregation

This is a credible and modern on-prem data fabric design.

The blunt truth is that the hardest part is not installing the components. The hardest parts are:

- enforcing Iceberg as the real contract
- committing to a central catalog and query plane early
- defining dataset ownership and metadata standards
- pairing governance metadata with enforceable authorization
- controlling multi-engine write semantics
- maintaining table health over time
- making lineage trustworthy through standardized publication
- stopping orchestration and governance layers from becoming chaotic

Get those parts right and the platform can work well. Get them wrong and you will just have an expensive collection of tools.

---

## 18. Source References


- OpenJDK JDK 21: https://openjdk.org/projects/jdk/21/
- Oracle Java SE Support Roadmap: https://www.oracle.com/java/technologies/java-se-support-roadmap.html
- Apache Iceberg documentation: https://iceberg.apache.org/docs/latest/
- Apache Iceberg REST Catalog spec: https://iceberg.apache.org/docs/latest/rest-catalog/
- Apache Polaris: https://polaris.apache.org/
- Ceph Tentacle Object Gateway documentation: https://docs.ceph.com/en/tentacle/radosgw/
- Ceph Tentacle Object Gateway S3 API: https://docs.ceph.com/en/tentacle/radosgw/s3/
- Apache Polaris GitHub: https://github.com/apache/polaris
- Apache Iceberg overview: https://iceberg.apache.org/
- Apache Iceberg multi-engine support: https://iceberg.apache.org/multi-engine-support/
- Apache Iceberg Spark quickstart: https://iceberg.apache.org/spark-quickstart/
- Apache Iceberg Flink integration: https://iceberg.apache.org/docs/latest/flink/
- Apache Airflow: https://airflow.apache.org/
- Apache Airflow prerequisites: https://airflow.apache.org/docs/apache-airflow/stable/installation/prerequisites.html
- Apache Airflow Docker guide: https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html
- Apache Atlas: https://atlas.apache.org/
- Apache Ranger: https://ranger.apache.org/
- Apache Kafka: https://kafka.apache.org/
- Apache Kafka Connect: https://kafka.apache.org/documentation/#connect

- Debezium: https://debezium.io/
- Iceberg Kafka Connect sink, one of the three writers compared in §6.4.7: https://iceberg.apache.org/docs/latest/kafka-connect/
- Apache Pulsar, evaluated as an alternative in §4.5.1: https://pulsar.apache.org/
- Pulsar architecture overview: https://pulsar.apache.org/docs/concepts-architecture-overview/
- Pulsar tiered storage: https://pulsar.apache.org/docs/tiered-storage-overview/
- Pulsar IO Debezium connectors: https://pulsar.apache.org/docs/io-cdc-debezium/
- Debezium Server, including the Pulsar sink: https://debezium.io/documentation/reference/stable/operations/debezium-server.html
- Trino documentation: https://trino.io/docs/current/
- Trino Iceberg connector: https://trino.io/docs/current/connector/iceberg.html
- Firebolt external data and Iceberg: https://docs.firebolt.io/performance-and-observability/iceberg-and-external-data
- ClickHouse documentation: https://clickhouse.com/docs
- ClickHouse Iceberg integration: https://clickhouse.com/docs/en/engines/table-engines/integrations/iceberg
- FreeIPA: https://www.freeipa.org/
- Keycloak: https://www.keycloak.org/
- Keycloak supported configurations: https://www.keycloak.org/server/supported-configurations
- PostgreSQL release documentation: https://www.postgresql.org/docs/release/
- Apache Maven release history: https://maven.apache.org/docs/history.html
- Podman releases: https://github.com/containers/podman/releases
- Docker Engine release notes: https://docs.docker.com/engine/release-notes/29/
- MIT Kerberos: https://web.mit.edu/kerberos/
- Prometheus: https://prometheus.io/
- Grafana: https://grafana.com/oss/grafana/
- Grafana Loki: https://grafana.com/oss/loki/